



Apollo: Performance Management Playbook

Course Overview

Course Overview

Hi. I'm Peter Kellner, and welcome to my course, Apollo: Performance Management Playbook. I'm a part-time docs writer for Microsoft, part-time author and consultant, part-time conference organizer, and full-time learner. This course is about leveraging the features of the Apollo GraphQL server and client to ensure your complete GraphQL solution maximizes performance to your customers while minimizing server costs and developer time. Some of the major topics that we will cover include, using the Apollo client cache to reduce the number and payload size of server requests, basic and advanced pagination techniques including infinite cursor based scrolling, configure Apollo's proprietary protocol to allow GraphQL queries to be processed over HTTP GET enabling content delivery networks, CDNs, to drastically improve the end-to-end performance of your apps, using the Apollo client to completely replace React's local state management libraries like Redux in your apps, and finally, how serverless and server-side technologies improve your overall app performance using the Apollo GraphQL technology stack. By the end of this course, you'll know what it takes to efficiently build and deliver high quality, easy to maintain, production ready applications that use both the Apollo GraphQL server and Apollo super fast client. Before beginning the course, you should have completed one of the GraphQL getting started courses or have equivalent experience. You should be comfortable programming modern JavaScript, as well as know the basics of React. Join me and learn all about how you can take advantage of the awesome features built into the Apollo GraphQL server and client with this course, Apollo: Performance Management, at Pluralsight.

Building a Simple and Performant Apollo Server

Learn Why Apollo GraphQL Is a Win for Both Client and Server Developers

Hi, I'm Peter Kellner, and welcome to my course, Apollo: Performance Management Playbook. The Apollo platform includes both Apollo Client and Apollo Server. They're separate, self-contained products, and you can use one without using the other, but they work together incredibly well, so we're going to



look at the workflow and performance benefits you'll get when you use them together. Through this course, we'll build both a GraphQL server and client starting in empty folders. This module, module two, is all about the server, and the next, module three, is all about the client. Following that, each module focuses on a specific performance enhancement the Apollo technology brings us. The server-based enhancements start in module seven, where we learn how to implement data loaders to solve the over-retrieving associated with nested GraphQL queries, also known as the N+1 problem. In module eight, we learn how to implement and customize server caching of data retrieved from external sources, and in module nine, the scaling benefits of serverless computing. The client-based Apollo enhancements start out with module four, where you'll learn how to leverage the Apollo Client cache, including data normalization, which can significantly improve browser-based experiences. In module five, you'll learn how to implement client-based pagination, including declarative techniques that increase code reuse. Module six focuses on techniques to completely manage Local State in React, and module nine implements, in detail, server-side rendering, taking advantage of the Apollo cache for persisting data between the server and client. The complete source code for all clips in this course can be found in the GitHub repo at the URL github.com/pkellner/pluralsight-apollo-managing-performance, separated by dashes. You'll find a folder for every course module. Then, in each of these module folders is the completed code for each clip. Drilling down one more level. Each clip is divided into a folder for our Apollo Server and a folder for our React website that includes the Apollo Client. Though most clips only modify one or the other, they are both brought along from clip to clip.

Building an Apollo GraphQL Server with Node Using One JavaScript File

Let's talk through the basics of what it takes to build an Apollo GraphQL server. I assume you already know the basics of GraphQL, so I'll keep this explanation short and focused on what differentiates an Apollo GraphQL server from a generic one. One of the many complaints you'll often hear about GraphQL is that there's a lot of indirection and boilerplate code you must create to make a GraphQL server work. I'm not going to tell you that's not true. However, it does not have to be complicated, especially with all the features Apollo has built into their server. The plan here is to start with the simplest possible GraphQL server we can put together using Apollo. Our server is going to be fully functional, and it's going to support query and mutations for both reading and updating data. What does it take to create a minimal Apollo GraphQL server? Let's build it first in pseudo JavaScript code so you can see the structure and the plan. The idea is that we'll create a file, `server.js`, that we'll launch with the Node executable. First thing is to import the Apollo Server itself. Then we declare our type definitions. I'll talk more about that momentarily. But for now, think of types



just like you would types in TypeScript, Java, C#, C++, or almost any language that supports object-oriented programming and classes. Following that, we'll declare our resolvers. Resolvers are how GraphQL takes incoming requests and resolves those requests into results. In this course, there are two types of resolvers we will work with. The first is query, which is the part of the GraphQL pipeline where a request sent in gets processed and the results are returned as the GraphQL response. The second, mutation, which again is the part of the pipeline where a request that contains their instructions and data associated with that gets processed and then also defines what gets returned as the response of that request. Typically, for something like an insert, part of the response would be the unique identifier for what just got inserted, typically an ID. If this were SQL, you can think of query as the SQL SELECT command, and mutation, the SQL INSERT, UPDATE, and DELETE commands. Our pseudocode here only supports queries. However, later in this module we'll add mutations, which is very similar to how we added queries right here. Finally, we have to create an instance of the Apollo Server we imported above. We do that with the JavaScript new operator and pass into it the type definitions and resolvers we just defined. This is also where we set up custom options for Apollo Server, things like cache policy, whether to enable Playground, whether to support CORS, and just basic configuration information. We'll take the defaults and not pass in any options. By default with Apollo, the cache is enabled. Cache in Apollo is one of the big differentiators Apollo brings that other GraphQL servers do not have. This cache is what helps make apps that work with Apollo Server perform really well. Finally, we just launch the server, and from here it just listens for incoming GraphQL requests, no different than how an HTTP server would listen for browser requests. That's it for pseudocode. In the next clip we'll build this out in real JavaScript.

Using GQL to Create Our TypeDefs

The final code we will build here in this clip can be found in our GitHub repo under the folder hierarchy module 2, clip 3, Apollo Server. Our starting point now is this Apollo Server folder. We've got our Visual Studio Code terminal prompt and VS Code File Explorer open in this folder. I assume you have npm installed on your computer, but if not, make sure you do install it, and have at least the most recently updated version of version 6 installed. Let's get started by first initializing Node and creating a package.json file for us. That's done by npm init space -y. Then, let's install our Apollo Server, npm install apollo-server. Now we're ready to create our JavaScript file. So let's create it right here in the root server.js. Before getting going, though, I realize that much of what I'm going to explain in the next few minutes will be things you already know. That is what I'm explaining is just basic GraphQL spec stuff and is not Apollo specific. Bear with me, though, as these basics are important to really understand how the performance benefits Apollo brings compare to plain old GraphQL. Back to our



JavaScript `server.js` file. At the top, get references to the named exports from our just installed `apollo-server`, `gql`, and `ApolloServer`. We'll go over by example the usage of these exports momentarily. Remember earlier we've said that for an Apollo GraphQL server, we need `typedefs` and `resolvers`. Let's start out by declaring our type definition using the named export `gql`. `Gql` uses JavaScript template literals as a parameter passing mechanism, that is, instead of calling `gql` with open and closed params like you would typically do to call functions, you use the template literal syntax. The syntax is very similar to just basic JavaScript literals that I'm sure you're familiar with. Something like `const fullName = `${first} ${last}` with the parameters in curly braces. To create our GraphQL `typedefs`, the syntax is `gql`` and then the full definition. This allows Apollo GraphQL to pre-process your `typedefs` in interesting ways. Back to defining our type definitions and what we put between those backticks. First, declare type `Speaker`. The syntax starts with `type`, followed by the entity name, `Speaker` in this case. Then in curly braces, we list our entity fields. The syntax is `field name colon field type`. These types are specifically GraphQL types. `ID!`, which is type `ID!`, meaning this is a required field and that it's serializable to a string, the idea being it can be used as a primary key to our full entity. It's not required, but it's helpful to have an ID like this. There are also other ways to define a primary key for an entity, and we'll talk about that more later. Then `first` and `last`, which are strings, and `favorite`, a boolean, which, if true, means this is one of our favorite speakers. That's it for our speaker definition. Next, we want to create an entity that's going to represent a list of speakers. Let's make it type `SpeakerResults` and have it for now just return one field, `datalist`. That `datalist` will be a list of speaker entities. That's what using the bracket syntax around a type represents. The final type, for our simple example, is the query type. This is essentially the root or top level of all queries that we can send to a GraphQL server. Let's give that just one query for now, `speakers`. Whatever we put after the colon following `speakers` is what the query `speakers` returns when we call it from a resolver. In our case, we're going to return data of type `SpeakerResults`. That's it for type definitions. In the next clip, we'll create resolvers that use these type definitions to return our query `speakers` we also defined in `typedefs`.

Building Field Level Resolvers for Our Schema

Resolvers, though not actually part of the GraphQL spec, are the typical way most GraphQL servers implement processing valid GraphQL requests. Specifically in Apollo, every field that we define in our `typeDef`, the schema that is, must have a resolver in order for it to participate in the rest of the GraphQL requests. Per the GraphQL spec, a GraphQL `typeDef` must include a root level type `Query`, which we have, and it has one field associated with it, `speakers`. We're back in VS Code, and our plan here is, just like we created a constant `typeDefs`, we'll create a new resolver object. While we are creating the resolver object, it will be helpful to be able to see the `typeDefs`, also known as the



GraphQL schema, so I'm collapsing here in this view the schema and showing it as an overlay. And we'll keep that up as we build our resolver so you can see it for reference. Let's start out by creating a JavaScript object, resolvers, and make the first property Query. In our typeDefs, or schema, we have one field defined under our root type Query, and that is speakers, so let's create a resolver for speakers. By convention, every field level resolver must be assigned to a function. These resolver functions take four parameters, parent, args, context, and info, and they must return a result as defined in the schema. Let's take a moment and define what those four parameters actually are. Parent is the parent resolver of the current resolver. In this case, speakers is the top level resolver, so it has no parent, and undefined will be passed in. You'll see in a moment as we move down our resolver for chain, parent will become very important. The second parameter is args, which, as you'll see shortly, is what gets passed into the request as essentially an argument passed into the GraphQL request itself. The third is context, which is typically some global configuration for your app, like a database connection string. Then finally, the fourth, info, contains more information, including things like the state of your application. Let's finish out our speakers field resolver. It wants us to return a type SpeakerResults, so let's create a JavaScript object to hold that result and assign to the datalist property an array of sample speakers. For this example, let me just hardcode 3 speakers, id: 101, David Jones, id: 102, Tom Lewis, id: 103, Doug Thompson. Then we just return our new SpeakerResults object from our speakers field resolver. Looking at our schema definition for SpeakerResults, we're going to need to define a resolver for that also, since that is what our speakers field returns. Let's create a top level type in our resolver called SpeakerResults and assign just one field to that, datalist. Then, just like the anonymous function we assigned to speakers, let's assign a similar function to datalist. That is, this will be our field datalist resolver. Again, the four parameters we pass into this function are parent, args, context, and info. This time, since we are being called from another field resolver, we need to use our parent reference to our speakers field resolver and return parent.datalist. Finally, we do the same thing for what our schema SpeakerResults returns. Notice it returns an array of type Speaker. This means we need to create a top level resolver type, Speaker, just like we did for SpeakerResults. This time, though, we need to create four field level resolvers, one for id, one for first, one for last, and one for favorite. Each one of these will take in the same four parameters that get passed in to all resolvers. Then each one will return the appropriate property passed into it from the SpeakerResults field resolver, parent.id, parent.first, parent.last, and parent.favorite. Looking at what we've got here for our resolver and comparing that to the schema on the bottom left, we've got a schema field definition for speakers; we have a resolver for that. And we have a field definition for datalist and a resolver for that also. And we have four schema field definitions for each speaker, and for each of those, we have a resolver. That's basically it. We've got our schema defined now, we've got our resolvers defined now. We just need to launch our Apollo Server and include our schema and resolvers in that launch definition. That's coming up in the next



clip.

Launching Our Apollo Server with Our Just Defined Schema and Resolvers

We now have everything needed to create an Apollo GraphQL server. We just have to launch it. Similar to how you might launch an express server in Node.js, if you've done that before, we do the same with Apollo. Let's continue where we left off in the last clip, with the typeDefs, that's the schema, and resolvers' completed. Then we write the code that launches the server that references both of those things. The typical steps are to first create an async function. We'll name it `apolloServer`. Then, in it, create a server instance by newing up our `ApolloServer` export we got from the `requires` on top. We passed to that our `typeDefs`, or schema and the resolvers we built. Then we launch the server listening at any port we want. I'll use 4000. The second parameter to the server is just a function that gets called when the server starts, closing our async `apolloServer` function, then executing it. That's it, we're done. Back to the command prompt. Enter `node server.js` and our server launches on port 4000. Bringing up a browser, browse into port 4000. We get a GraphQL playground, which lets us send post commands to the GraphQL server we just built. Using IntelliSense, we can create our query. It's just `query, speakers, datalist`, then the four fields and then launch it. Sure enough, there are three speaker records just as we planned. Assuming you've used Apollo GraphQL before, you're probably wondering if I really needed to create resolver for speaker results and for every field in the speaker type. You'd be right in guessing, no, I did not. I did create them because I never like to take for granted what Apollo's going to create on my behalf. In the next clip, we'll take a look at automatic resolver creation in Apollo.

Understanding Apollo Server's Automatic Resolver Creation

We are again looking at our Apollo GraphQL server JavaScript. That is the single file that we launched with Node. In a previous clip, I hinted that maybe all we need is the speakers resolver under Query and that the other field resolvers for `SpeakerResults` and `Speaker` are not necessary. How could that possibly be? Well, turns out that the Apollo GraphQL server, when it processes a GraphQL request like I've got here for a speaker record, follows a flow chart for how to resolve that request. That flow chart includes what to do if there is no specific resolver found, rather than just returning an error. Let's look at that flow chart. It starts out by asking the question, does that parent argument have a property with the resolvers exact name? The easiest way to follow this chart is with a specific example. Let's look at the speaker type with its four field resolvers, `id`,



first, last and favorite. I've taken the liberty to shorten the code with some ES6 syntax. When we sent our request from the GraphQL Playground, we asked for the speaker id field. Thinking like the Apollo client that's processing this request, we are here in the Speaker type of the resolvers list, and the question is, does the parent argument have a property with resolver's exact name? Let's parse that statement. The first question is, what is the parent argument that is being passed into this id field resolver. Since SpeakerResults returns an array of speakers, that's the datalist, parent is just one speaker record. So does an individual speaker record have an id property? The answer is, yes, it does. We go back to our flow chart and follow the yes arrow. Is that property's value a function? Well, in our case, it most certainly is. It's right here. The function is defined and returns parent.id, which, following our resolver back, is the id from the original datalist. Okay, so what happens if we were to remove the speaker id field resolver? Let me just comment it out for now. Back to our flow chart. Is that property value a function? No, not only is it not a function; it's not even defined. That's still a no. Going the other direction to the no on the flow chart, we return the property's value, which in this case is parent.id, exactly the same as what our resolver would have returned had we written it. Maybe you're wondering, why bother with the id, first, last, and favorite resolvers? That's easy. Let's say you want to always return first and last as uppercase. Let's go back to our server.js file and add that to our resolver. All we have to do is add to our return of the first and last field resolvers the function toUpperCase. Before heading back to our playground, though, and re-executing our GraphQL command, let's go to the terminal prompt, stop our Node server, and let's install nodemon so that future changes we make in our server.js file will automatically restart our GraphQL server. Let's add another command line to our package.json file, "start:" nodemon server.js, and then run it, npm start. Back to our Apollo playground from the last clip. Re-executing the GraphQL request, our results are all uppercased. Say we wanted to create a combined first-last field. To do that, we of course have to add it to our schema. Otherwise, GraphQL will not even consider it valid. So we're back here in server.js looking at our schema. So let's add it as a String type. Then we just add another resolver, firstLast, give it our standard four resolver parameters, and then return first and last concatenated together with the JavaScript literal syntax. Heading back to the playground, adding our firstLast field, re-executing the query, and now we've got it. In addition to our other fields, we have a new one, firstLast. Just a side note, notice that the names in the new firstLast field are not uppercased. That's because our flow is that each field resolver is executed independently, and what first and last return have nothing to do with what firstLast returns. Back to the original question, do we need any of these field resolvers? Well, we commented out id and it still worked. Let's comment out all of them, including firstLast, and let's remove firstLast from our schema. Now back to our playground, remove firstLast, execute the GraphQL. Sure enough, it all works as we would have expected. No errors. Our flowchart worked. Following the same logic for the resolvers type SpeakerResults and the field level resolver datalist, we can remove that



code as well. We now only have a query type with the speakers field level resolver, not other field resolvers. So back to the playground. We run the query and everything still works. Auto-resolver creation from the Apollo client just saved us a lot of code that we otherwise would have had to write. Mission accomplished.

Integrating REST As Our Speaker Data Source With json-server

Our goal in this module is to build a GraphQL server as simple as possible, making the fundamentals of process flow inside the server very clear. To that end, we're doing it with one minimal file, `server.js` with the simplest constructs possible. So far, we've only got one of the GraphQL root query types to find, query, that lets us get data from our GraphQL server, but not update it. To write data to our GraphQL server, we need another root query type and that one is mutation, and that's what we're going to focus on for the rest of this module. To support data being updated, we need some kind of persistent store that can handle updates. This could be something like local storage, it could be a database, or in our case, it's going to be a REST server. I find the best thing for courses like this is to use the npm package `json-server`. It allows for running locally in a development environment, and in my experience, perfectly mirrors the REST standard. The benefits are your development environment does not require you to have access to a production server or some hard to maintain copy of it, you can stay ahead of the server development team because simulating feature updates to the REST server is easy, you can be reasonably guaranteed that your REST calls are standards compliant because `json-server` implements the REST standard very well, and it's easy to work with large data sets since the source for `json-server` data is just a local file. Let's now integrate `json-server` into the project we've been working on. First, install `json-server` `npm install json-server` `save-dev` since we're only going to use this in development. While we're here, let's also install `npm-run-all` to give us a convenient way to launch our `json-server` when we run `npm start`, details coming up on that next. Then let's create a source for our data `db.json`. The format `json-server` expects is in json notation, it's an object that for each REST URL, there is an attribute, `speakers` in our case, with an array of speakers as the value. We've been using a JavaScript array of three speaker names up until now. That was convenient for embedding in our small JavaScript file, but now we can bring in a lot more data. And later in the course, when we get into things like infinite scrolling, we're going to want a lot more data. Over the past 15 years, I've organized a pretty large event here in Northern California called Silicon Valley Code Camp. We're going to use speaker and session data pulled from many of those 15 years, hundreds of speakers. Our `json-server` is always going to read and write to the file `db.json`. However, I've included three additional files here in the root of the project, `dbLarge.json`, `dbSmall`, and `dbTiny`. `DbTiny` just has five speaker records in it making it easier to work with in many



cases. We're going to start out by copying dbTiny to db.json. Notice also, we've got some extra fields. In addition to id, first, last, and favorite, we now have Twitter handle, bio, and company also. We'll include those later when we bring in some nice CSS styling. For now, though, let's keep it simple while we are doing the server side of Apollo.

Integrating and Launching json-server with Our Apollo Server

Now that we have our JSON server files ready to go, let's update our package.json so that we can launch it with npm. Start out in the scripts section and add a launch command for JSON server; json-server db.json, add a delay of half a second or 500 ms, and have it start on port 5000. We could always start our JSON server in a separate terminal window by entering a command now, npm run json-server, and then separately start our GraphQL server like we did before npm start. But it's a lot more convenient to have our JSON server start when our GraphQL server does. For that to happen, let's rename our current start to run-server, then make a new start command using the npm run all package we installed earlier. The new command is start npm-run-all --parallel json-server run-server. One small change, though. To our node daemon, add --ignore db.json, so every time we update our db.json file with a PUT, POST, or DELETE, the node server doesn't restart again. Now, add our terminal prompt npm start, and we've got both our GraphQL and JSON server running. Notice the launch message on JSON server showing us the REST resources, just speakers at the moment, localhost 5000/speakers. Doing GET, PUT, POST, and DELETE to that endpoint will update our db.json file appropriately. Heading over to our Apollo GraphQL server.js file, let's update our speaker's resolver to get data from our new REST server instead of the static JavaScript array we were using before. Two basic choices here, as I see it. The Apollo way would be to incorporate a comprehensive package they've written called REST Data Source. Here are the docs for this, as well as an example. If you are bringing in a REST API with a large surface area, using the REST data source can be very helpful. There are, however, two reasons I'm not going to use it in our example. The first, and most important, for a single simple REST service, like just speakers, it's just easier to inline it rather than bringing in the complexity of REST Data Source. Second, it is only available for Apollo at the moment as a JavaScript class, and I prefer to only use functions in my apps and only classes when absolutely necessary, which is practically never. The other way is to make our REST calls directly from our server.js file resolve ers. That's what we'll do. Personally, I like using Axios, which is widely used and accepted in the open source space. So install the npm package Axios, npm install axios, import it at the top of our server.js file, then update our resolver to be async with async functional syntax, remove our old implementation that just returned the static array. Before defining speaker results, make an async call to Axios, const response = await axios.get, our REST URL, and then return our



response.data from this method. That should be all we need. Bring up our GraphQL Playground, re-executing our speaker's query, and it works just like before. But now we have five records.

Inserting, Updating, and Deleting Speaker Data with Mutations

Now that we've got a REST service attached to our Apollo GraphQL server, let's update our server to handle three other REST protocols besides get, that is put, post, and delete. The place to start is our schema. Let's add another root type, Mutation. Let's create three different mutations, one each for put, post, and delete. First put. Put is the REST equivalent of update in SQL. We could write a resolver that updates the entire speaker record, but for now let's create a mutation method that takes in a speakerId and toggles the SpeakerFavorite property, toggleSpeakerFavorite. It's going to take one input parameter, speakerId, which is an int!, the dollar sign meaning it's required, and let's have it return the updated speaker record. Scrolling down to our resolvers, let's add a new root type here below query, Mutation that is. And let's define the resolver method as async, and just like the schema entry, name it toggleSpeakerFavorite. Like all resolver methods, it takes four parameters, parent, args, context, and info. All we really care about is args.speakerId, as that's what's required to be passed in to this Mutation method. First, just like we did in the speakers query, make an async call to axios.get, but this time include the speakerId concatenated at the end of the URL. We don't need to do any special error checking, because if the axios.get fails, a 404 Not Found error gets thrown, and that will propagate through the server and be returned to the GraphQL client. Create a record with the favorite status toggled using the spread operator. Then await the axios.put, passing in first a put URL, that's the same URL as we used for the get, and then the new speaker record with the favorite field toggled. Finally, return the updated speaker record. Let's do something similar for inserting and deleting speakers. First, go back to the schema above. For inserting, we need to pass an input type in, so we have to define that first, just input SpeakerInput and the three properties, first, last, and favorite. Then at the Mutation method, addSpeaker, pass in the named argument, speaker and SpeakerInput, as the type. Finally, return the speaker record, which will include the generated primary key, id. While we're here, let's add the definition for the mutation deleteSpeaker, just deleteSpeaker and passing in speakerId, and then returning the speaker record just like before. That is the speaker record that got deleted. Now let's add the two resolver methods to match up with the Mutation schema definition. Similar to the async Mutation, toggleSpeakerFavorite, let's create both the addSpeaker and deleteSpeaker methods, addSpeaker first with the four parameters we always pass into resolvers, and then deleteSpeaker same four parameters. For addSpeaker, first destructure the arguments passed in from the GraphQL command, getting us first, last, and favorite. No id, as that will get



generated by the insert itself. Do a quick check to make sure that there is not already a speaker with the same first and last name. Do that by first getting a list of all the speakers, then searching that list using JavaScript array find. If a duplicate is found, then throw a special Apollo defined error called `UserInputError` with a message. We do need to scroll up and add that as a named export from our Apollo library. Then finally, call Axios with `axios.post`, passing in our first, last, and favorite attributes. Lastly, return the return record back to the GraphQL caller with the full new record, including the new primary key ID generated. For delete record, this is much simpler. First, get a URL that includes the speakerId. Then do `axios.get` with that URL to retrieve the record you want to delete, then `axios.delete` to delete that record. Lastly, return the data that we got from the original get. If the get fails, then we'll get a 404 thrown, so we don't really need to do any other error checking. That's it, we've now got four REST verbs implemented. Get we did previously, and post, put, and delete we just completed. Coming up, we'll test our new mutations in Playground.

Run Our New Insert, Update, and Delete Mutations in Playground

Now that we've got our simple Apollo GraphQL server implemented supporting speaker inserts, updates, and deletes, let's form some GraphQL queries in Playground and see them in action. So here in Playground where we have a query that was the root type we used before, let's instead use mutation. The method name, then, is `addSpeaker`. Pass in the speaker type with the properties first, last, and favorite, setting favorite to false, then returning the full record id, first, last, and favorite. Executing that, we get our new record returned with an id. Success. Now let's test our `toggleSpeakerFavorite` mutation. Change `addSpeaker` to `toggleSpeakerFavorite`, then just pass in the single argument `speakerId` set to the id we just got back. Then return all our properties. Keep an eye on the output as we press the Execute button. Notice favorite is set to false as that is what we added previously. Executing our `toggleSpeakerFavorite`, now it's true. Executing it again, it goes back to false. Success again. Finally, let's test `deleteSpeaker`, changing `toggleSpeakerFavorite` to `deleteSpeaker`. Press the Execute button. We get our deleted record returned. Pressing Execute again, 404 because the record no longer exists. That's it for creating mutations. We've got the full REST complement, get, put, post, and delete, working now. That's it for our module on building a simple performant Apollo Server. The big takeaway is if you use the Apollo Server, you'll have a big step on getting your app ready for production. Features like automatic resolver creations, defining schemas with template strings are examples of how Apollo makes things easier, better, and faster. We took a very shallow look at these features. In the upcoming modules, we'll expand on them, as well as introduce more.



Building a Simple and Performant Client App

The Apollo GraphQL Client Introduction

Hi, I'm Peter Kellner, and welcome back to my course covering Apollo performance management. This module is about building a React app using the Apollo GraphQL client. In the previous module building an Apollo GraphQL server with Node.js, the server we deployed will be what we will connect to going forward. The React client we will build in this module connects to the Apollo Client using the best and latest features the React team has built. This includes extensive use of both React hooks and the React context APIs. More specifically, we'll be starting out by setting up the architecture of our React app, such that when the app first renders, an Apollo Client is configured and then with little extra work in each component, will have access to that Apollo Client. We'll start out learning how to use these `useQuery` custom hook built into the Apollo Client giving us access to all queries exposed by the Apollo GraphQL server. Then, after learning how to use the Chrome Developer extension for working with the Apollo Client, we'll learn the details of using that `useMutation` hook, which gives us a way to insert, update, and delete data exposed from our GraphQL server. Along the way, we'll talk about how just using the Apollo Client and Server gives us benefits derived from the cache built into a React client. For each clip in this module that has code, there's a subfolder in our GitHub repo associated with this course. That folder has two subfolders. `ApolloServer` and `ApolloClientReactApp`. That `ApolloServer` folder is always the same code, and it's what we finished within the previous module. To launch it, just launch a terminal prompt there and issue the command `npm install`, and then `npm start`. If you have questions on the server configuration, I suggest reviewing the last module.

Creating a Simple React App with Next.js

There are multiple ways to create a React web app starting from an empty folder. The Facebook React Team has a web page listing several of them. Under the page header, Recommended Toolchains, the first one they list is Create React App, which the React Team wrote themselves. Personally, I find the second one listed, Next.js, usually referred to as just Next, is much easier to use, easier to start new projects with, and gives you a lot more flexibility when deploying, including server-side rendering and static site generation. It also gives huge benefits for web performance scalability and SEO. Coming up, I'll go through how to set up a project with Next and some very basic functionality. However, if you want to learn more, I have a full course on Next titled Building Server-side Rendered React Apps for Beginners here on Pluralsight. Let's dive right into creating a React web app in the empty folder we created back in the second clip. I've opened Visual Studio Code right in that folder. First, open a terminal prompt there, then



to start, let's initialize our node package manager by typing `npm init -y`. All that's necessary to create a React web app with Next is to install three packages. Let's do that, `npm install react react-dom next`. We do need to add some script commands to our package.json file, so let's do that also. Add the command `npm run dev`, giving you a full interactive build experience, including live coding in advanced debug features, `npm run build` to create an optimized and minified production build of our app, and `npm start` to launch that production build. Now all that's left to do is create a simple home page for our app. We do that by creating a folder named `pages` in our root, and in that folder, a JavaScript file, `index.js`. In that file, we'll create a simple React functional component, including JSX for rendering, adding it, `const IndexPage = a lambda function`, then return `GraphQL is here to stay`, and finally, `export default IndexPage`. Now, going down to the terminal command, running `npm run dev`, our server launches. Open a browser at port 3000. It works as expected. GraphQL is here to stay! In the next clip, we'll import the Apollo GraphQL library, and we'll add that to the client we just developed in React.

Adding the Apollo Client to Our React App

Now that we have a simple Next app coded, our next step is to install what's necessary so that we can use the Apollo GraphQL Client in it. We want to make sure we set it up in a way that when React renders a page, that before any components are rendered, the Apollo Client is initialized and that every component loaded thereafter has access to that Apollo Client instance. Let's start out by installing the necessary packages, `npm install @apollo/client graphql bootstrap`. Before updating our `index.js` file to include what's necessary to access our GraphQL Apollo Server, we need to create an `ApolloProvider` using the React Hooks Context API. Basically, this is an API whose purpose is to allow us to create a global-like configuration that we can include in any component. In this case, the folks at Apollo have done all the hard work for us, and all we have to do is reference that using named exports from that single `ApolloClient` import. If you do want to know more about how Context works or even use the Context API in your React app and other places, I recently released a 3-hour course here on Pluralsight titled `Using React Hooks`. The third module, `Using More Hooks`, explains the basics of the Context API, and the last module, `Using Context and useReducer to Make a Redux-like Global State`, goes into a lot more detail and would give you insight into how the Apollo team built and architected both the `useQuery` and `useMutation` custom React Hooks that are the center of the Apollo Client implementation. We'll be integrating `useQuery` into our React app shortly. Back to our problem at hand. We need to establish the `ApolloProvider` that using the React Context API gives us access to our Apollo Client Context in every component of our React app. Next.js gives us a very convenient way to do that. All we have to do is create a special file in our folder `pages` named `_app.js`. This file will essentially run before every invocation of Next routing pages



in the pages folder. What I mean by that is that the way Next resolves routes coming into the React app is that any URL you access the React app with is parsed, and what comes right after the domain name gets routed to a file in the pages folder with that same name. Say, for example, our site is hosted at myconference.com. If we browse to the URL, by default it will be routed to a file named `\pages\index.js`. If, however, we browse to `myconference.com/speakers`, we will be routed automatically to `\pages\speakers.js`. My point here is that any file we create in the pages folder will be the target of a URL route, and every one of those files, thanks to the Next architecture, first runs this special file, `_app.js`, in our pages folder. So what do we put in this special `_app.js` file? Just a simple React functional component. Let's create that. Export default function `App`, it receives two destructured prop references, `Component` and `pageProps`. Then, using JSX syntax, it returns the passed-in `Component`, passing in it those `pageProps`. What we've got here now is an `_app.js` that effectively does nothing other than wrapping our `Component`. If this looks familiar, it's basically the Higher Order Component Pattern, or HOCs. In my course here on Pluralsight, *Designing React Components*, the fifth module is all about HOCs, and if you want more background on them and how to use them, I go into a lot of detail, so check that out. The next step is to wrap that returned `Component` in an `ApolloProvider` that will give all our components access to an instance of our Apollo Client that we will instantiate here in this file. For this to work, we do need to import some named exports from our Apollo Client library. That is, let's import `ApolloClient`, `ApolloProvider`, `InMemoryCache` from `@apollo/client`. Then let's use the first one, `ApolloClient`, to create an instance of our `ApolloClient` just before we return our JSX, `const apolloClient = new ApolloClient`. Then we set two of the many options, `uri`, which points to the Apollo GraphQL server we built earlier, and `cache`, which is set to `InMemoryCache` for now. Finally, instead of directly returning our `Component` with its `pageProps`, let's wrap our `Component` using the named export, `ApolloProvider`, which happens to be a component. For this to work, we need to assign our `ApolloClient` instance to our provider's `client` attribute, `client={apolloClient}`. We've now effectively given every `Component` of our React app access to our `ApolloClient` instance. In the next clip, you'll see that in action wrapping our Components..

Adding the Apollo Client `useQuery` to Our React App

Everything we've done so far in this module has prepared us to be able to very simply build a web page in React whose purpose is to send a query to a GraphQL server, then render the results of that query to a browser. Let's get right to it by modifying our `index.js` page to do just that. First thing, we need to import what we need from the Apollo client node package import `gql` `useQuery` from `@apollo/client`. In our functional component, let's define the GraphQL query we want to send to our server using `gql`. Going back to our playground from a previous clip, let's grab the query itself and bring it back to our `index.js`, define it as `const`



GET_SPEAKERS = gql` and then paste in that query. Now with useQuery, the React custom hook we imported above, let's call it with that speaker query we just defined. It returns using destructuring syntax loading, error, and data. We call useQuery with our GET_SPEAKERS definition from above. Then after that, we check some statuses, and finally, we render the data. First, check to see if the data is loading, then check to see if an error returned if neither render the data returned from our GraphQL server. Before doing that though, let's take a step sideways and talk about what our plan is for the rest of this module, and also for the rest of the course. By the end of the course, we will have developed using many capabilities of both our GraphQL server and client, a speaker page of a complete conference site. That page will have capabilities that include multiple types of pagination, local and remote caching, local state management, and much more all based on Apollo Technologies. Our final page is going to look like what I've got here. For this module, we are just going to build out a simple speakers list with just first, last, and favorite. That favorite icon is going to be clickable to demonstrate toggling the speaker favorite. We're going to have a Delete button to remove the speaker and Insert button in our toolbar to create a new speaker. The rest of the page we'll build in later modules. Keep that in mind as I explain the bootstrap we are using because you may notice I have some extra containers and rows that are going to make our life easier later when we bring in those other parts of the web app. This page is based on a customized bootstrap theme that we're going to need to include in our app. As we build our renders in our React components, I'll use bootstrap and other custom classes to make things look good, but I won't spend time talking about them because one, it's not a focus of this course, and two, the CSS was designed on a static web page and I just pulled the classes and moved them over as appropriate. We do need to include the necessary Bootstrap and customs CSS and some fonts, and the easiest way to do that in Next.js is to create an _document.js file in our pages folder directly, just like we created an _app.js file in the previous clip. In the nextjs/docs, there is a skeleton _document.js file they provide for us that lets us include our own head element in every rendered page. I've got that copied here. Let's reference our CSS assets, but first, let me drag in three CSS files into our public/css folder. Public is where next.js loads files directly as if they were in the root of the web folder. These files are just Bootstrap itself and two customs CSS files. Now the references. First, a link tag that pulls in bootstrap, then two different includes for font awesome that's going to give us nice symbols and icons. Next, some Google fonts, and finally, our custom css/style-speakers. Now we have our styles, back to rendering our Speakers page index.js. Bear with me and just ignore my extra CSS classes I'm throwing in for styling. Start out with, first, a Bootstrap container div, then add a div with the Bootstrap row class, then finally a helper div for the class fav-list, then render our data using a JavaScript expression. That expression is data.speakers.data.list.map, then the mapping function, destructuring the props id, first, last, and favorite, and from that, return a speaker box row and design the attribute key for React uniqueness. Then let's create two columns, one will hold the speaker name class col-sm-7 for sizing, and



the other the speaker favorite status and that will be a col-sm-5 just a little less wide. Then, if the favorite status is set to true, put a hard icon solid, set it to hollow, follow that by the word Favorite, close the divs, and we're done. One thing to mention before browsing to the page, anytime you update either `_app.js` or `_document.js`, I recommend restarting your development server, that is `npm run dev` to make sure `next.js` loads properly. Now passing to the page and we've got it, all five speakers rendered from our GraphQL server using the `useQuery` custom React hook included in the Apollo GraphQL client.

Install and Use the Apollo Chrome Debugger Extension

Let's now add the Apollo GraphQL Chrome debugger extension to Chrome. As of now, as this course is being published, if you want to use this extension, it's strongly recommended you use Chrome, and not any other browser. The Apollo team does have other browsers in the works, but for now, Chrome is what is recommended. The fastest way to install an extension in Chrome is go to the URL chrome.google.com/webstore, search for Apollo, then click on Apollo Client Developer Tools, and install it. Going back to our running React app, let's bring up our Chrome DevTools, either F12 or from the top right menu, More Tools, Developer Tools. You should now have a tab titled Apollo. Tabbing to that, you immediately see the GraphQL query we sent to our GraphQL server. Using the Run button in the toolbar, you get the results of the query. You can change the query right here. Say I wanted to remove the first name, I could just delete it, run the query again, and our results would reflect that. Also, built into the tool is a full GraphQL Explorer giving you even more details. In the left side menu, we can move down to Queries. There you get a list of all the queries this page sent to the server in all your components. We only sent one, so it's a very short list. Next is the Mutations, or updates. We have not sent any of those, so the list is empty. Finally, the last and most important, in my opinion, is Cache. So first is it cached the entire `ROOT_QUERY` for speakers. In addition, it normalized, which we'll talk more about later, each speaker record and cached those independently. The benefit of that is if, say, later we build some other GraphQL query that references any one of these speakers, the Apollo Client remembers the primary key of the speakers list and can individually go get those records from Cache instead of having to go to the server again and re-fetch them. Let me get a little ahead of myself here by going back to our `IndexPage` and remove the speaker id from all places it's referenced. First the GraphQL query, then the props list, fix the unique key to be a concatenated first and last string, and finally remove the second column it's displayed in. Back to the browser, refresh the page, and now look at the Apollo Cache. Notice now we just have the `ROOT_QUERY` Cache and no individual speakers. All the performance benefits that individual speakers cache would bring are lost. That's because Apollo automatically recognized id as the primary key and took care of caching that for us. Coming up in the next clip, we'll look at mutations and you'll



see that cache in use.

useMutation for Updating a Speaker Record

You've seen the `useQuery` custom React Hook, which is part of the Apollo Client library in action. That is, that's what we use to query our Apollo GraphQL server. Now let's bring in `useMutation` and use that Hook for toggling the favorite icon for an individual speaker. We're here in the GraphQL Playground looking at the mutation named `toggleSpeakerFavorite` that we developed and tested back in the last module. Notice we hard-coded in a `speakerId` to test this. In our case, we need to pass in the `speakerId` based on the current speaker row in the UI. Let's pick up this code and head back to our `index.js`. Just like we defined `GET_SPEAKERS`, let's do the same for our toggle, `const TOGGLE_SPEAKER_FAVORITE = gql``, then define the mutation. This doesn't quite work, though, because we need to pass in the `speakerId`. This is where our `gql` template literal helps us. The way to add parameters to a mutation is to name that mutation. I like to make it PascalCase instead of camelCase. In other words, first letter capitalized, and then specify the incoming parameters, parameter name, colon, GraphQL type, in our case, `speakerId: Int!`, meaning required. We then need to pass that template variable into the call itself by replacing the number with `$speakerId`. Then moving down to our function, we need to reference this `TOGGLE_SPEAKER_FAVORITE` with `useMutation` just like `GET_SPEAKERS` we referenced by `usequery`. The syntax for that is `const ToggleSpeakerFavorite = useMutation(TOGGLE_SPEAKER_FAVORITE)`. Notice that `useQuery` is returning an object that we're destructuring to properties, and `useMutation` is returning a tuple with the first value being the function that we call to execute the mutation. We do, of course, have to add `useMutation` to our imports from the Apollo Client. Scrolling down to our favorite icon, wrapping our span with the `className` `action`, then adding the `onClick` event to our span. Just call the event `toggleSpeakerFavorite`, pass an object to it with the attribute name variables, and list our values. All we want to pass in is `speakerId` in this case, convert it to a number, and we're all done. We've got a click event associated with our `SpeakerFavorite` icon that hopefully will toggle our favorite value from true to false or false to true. Let's try that. Back to our app in the browser, refresh the page, click on the first row Favorite icon. Sure enough, after a short delay from the REST service, it toggled. You're probably not surprised that it worked, but you really should be. Why? All we did is we called the method returned from the `useMutation` Hook, `ToggleSpeakerFavorite`, and technically we didn't do anything to update our React UI, but somehow the favorite icon toggled. The answer is in the Apollo cache. Let's bring up the Chrome debugger and then the Apollo extension tab, navigate to Cache, and let's select a normalized cache result for the first row, `Speaker:14918`. Keep an eye on the favorite status. When I click on the icon, it changes. That change propagated through the full Apollo Client, and ultimately updated our React UI without



having to use the `useMutation refetchQueries` option. In other words, without the Apollo Client and Server working together, we would have had needed to refetch our original speakers query to get the UI to update. The module following this one takes a deep dive into what actually goes on to make that happen. For now, it just works as we hope, even though we really should not have expected it. Next clip, we'll add the speaker delete, also taking advantage of the Hook `useMutation`.

useMutation for Deleting a Speaker Record

Let's now add the capability to our speakers list UI to delete an existing speaker. From the previous module, just like we discussed in the last clip, we created and tested a GraphQL mutation for deletion. The mutation is almost identical to the `toggleSpeaker Favorite` mutation we just added. Let's go back to our `index.js` where we defined the `toggleSpeakerFavorite` query, and make a copy of that and modify it to become the `deleteSpeaker` query. I'll just copy that, and then rename `TOGGLE_SPEAKER_FAVORITE` to `DELETE_SPEAKER`. Moving down to the function execution, let's add another call to `useMutation` to expose a `deleteSpeaker` method we can call, `const deleteSpeaker = useMutation(DELETE_SPEAKER)`. Scrolling down to the render after the favorite icon, add a new span that encloses a trash icon using the `fa-trash` and `red` class. Then again, just like `toggleSpeakerFavorite`, create an `onClick` event here. Then call `deleteSpeaker` with the `id`. Are we done? Let's check. Refresh the browser. We've got our trash icon. Let's delete the first row by clicking on the trash icon. We get our delay, but it appears nothing happened. Did it? Let's refresh the browser. Sure enough, it did work. The speaker is gone. Turns out the Apollo Client does not do anything for us regarding the cache, and that cache is what's reflected in the UI, that is only update mutations are helped by the Apollo Client itself. Neither delete or insert as we'll see in a minute, do any behind-the-scenes operations on the cache. It's clear to me why the Insert's done, and that's because it's not obvious where you put the new record, top of the list, bottom, sorted someplace, who knows. Certainly not the Apollo Client. It does seem to me, though, that Apollo could delete the cache record, but I'm sure there's some corner case that they're avoiding, probably for good reason. How do we fix this? Easy. Just add to our mutation another property, `refetchQueries`, that takes an array of queries we've defined with CQL, and then when the mutation completes, those queries are run again. Refreshing the browser, press the trash icon on the top speaker, and after a short delay while the REST call completes, the UI updates correctly as `useQuery` re-retrieves the current speakers, and the one we deleted is gone. Next up, insert speaker.

useMutation for Adding a Speaker Record

For inserting a speaker, we're going to need a little UI work. We'll need to create a



toolbar on the top of the page and also create an input form to enter new speaker details, along with a Save button, of course. Though I hesitate to just paste in a bunch of code, I think in this case it's okay. Let's start out on our index page. That's where we've got all our UIs so far. Let's wrap our existing complete UI in a React fragment, then above our existing UI let's add a new component that we'll need to create, Toolbar. Let's make the spec for that Toolbar to have just one parameter, a function that the Toolbar calls that passes in the first, last, and favorite values, so we can process and add speaker mutation, which we'll also have to define. Now for the Toolbar itself. Let's create that in a new components folder and just call it Toolbar.js. The functional signature is simple, `const Toolbar =` the Lambda function with the receiving prop, `insertSpeakerEvent`, that's actually a function. And then everything else is about supporting a click event on a Save button that the user clicks, which calls `insertSpeakerEvent` with the three speaker attributes, first, last, and favorite. That all said, let me just bring in all the code now and update the imports. You'll need to install the npm package `reactstrap`, because that contains the form library we are using, and it works on top of `bootstrap`, which we already have installed. You can see here in the `handleSubmit` event we are calling our `insertSpeakerEvent` that was passed in as a property. Back to `index.js`. Let's import our new Toolbar component, then do our `insertSpeaker` work. Looking back at our speaker `insertMutation` we defined in a previous module where we wrote our Apollo GraphQL server, it's not that different from the `toggleSpeakerFavorite` or `deleteSpeakerMutations`. The passed-in parameter, instead of just being an `int`, is a speaker record with three properties. Let's create that mutation in code. Back to `index.js`, `const ADD_SPEAKER = gql``, and now the three parameters instead of just one, first, last, and favorite, Then call our mutation directly with the attribute speaker instead of `speakerId`, and pass an object with first, last, and favorite, instead of just `speakerId`. Like with all mutations, we need to return values, and that's `id`, first, last, and favorite. Next, we need to call our `useMutation` React Hook for `addSpeaker`. We just add `const addSpeaker = useMutation(ADD_SPEAKER)`. Then in the function returned from the Toolbar, make the call to `addSpeaker`. Pass in the three variables, first, last, and favorite, remember, we got those from what the user typed into the form, and we're almost done. All we have to do now is exactly the same as we did for `deleteSpeaker`. We need to call `refetchQueries` with our `GET_SPEAKERS` query so the UI updates. Crossing again to our page, clicking on the Insert Speaker button, up comes our Toolbar form. Type in the name, click the Favorite checkbox, then Save. Sure enough, there it is. Works as expected. We've reached the end of our Apollo Clients module. We've just scratched the surface of what's available and what you need in order to successfully build performant apps using the Apollo Server and Client technologies. What's important is that we have an architecture that allows us to share the Apollo technology throughout our React app. So you've now got a basic understanding of how `useQuery` and `useMutation` work, and those combined give the ability to read and write to a GraphQL server from Apollo Client. You've got the big picture of how cache in Apollo can help your apps be more performant, but just from the basics, only to a



certain point. In the very next module, we'll take a deeper dive into the Apollo Client, specifically around how cache helps our apps. We saw how update mutations have a big performance benefit from using the Apollo Client, but insert and delete were not helped. You'll learn how, with your own custom code, your inserts and deletes will have the same performance benefits the update we coded in this module has.

Learning Apollo Client Caching Techniques

Why Use Cache in the Apollo GraphQL Client?

Hi, I'm Peter Kellner, and welcome back to my course covering Apollo performance management. In this module, we're going to focus on the data caching in the browser with the Apollo Client instantiated. Why do we need the Apollo Client cache when building our React apps? A good way to answer this is to think about it from either the server or the client perspective, that is, the company hosting Apollo Server, often a cloud provider like Azure or AWS or the user sitting in front of a web browser running an app, in our case, a React web app. What's good for the server? Fewer round trips between the browser and the web server, since a lot of the data is stored in the Apollo Client cache. Often we don't need to make extra server requests for more data. Less computational load required. Think about sorting data. So you want to sort a list of hundreds or even thousands of items? You could do that easily by sorting just the data in the Apollo Client without ever contacting or loading the server. Less network traffic. That's practically the same as less round trips, but worth mentioning. And, of course, all this leads to lower costs. If you are paying a cloud provider, the less work you have the server do, the cheaper it is. So, what's good for the user, that is the user at a web browser? For one thing, the user interface is more responsive. If the data is coming from the cache instead of from the Internet, that's a win. That leads to better performance on slower connections, since we don't need to get the data from the server. It allows us to build an optimistic UI. Think of an optimistic UI like on Facebook when you like a post. The Like button highlights right away, even though the server may not update for a few seconds. And finally, it allows the user to use the CPU and memory they have on their device. Things like sorting can be expensive on a server, as those servers are having to do that for thousands or even millions of users. If you sort data from your local cache, you have 100% of your device to do that work, and that can be blazingly fast. Just a brief word on the ApolloClient configuration. In the last module, we initialized the Apollo cache to use the provider `InMemoryCache`. Basically, that uses the local memory in the browser, and a big benefit is it has no external dependencies. Regarding configuration parameters, we took all the defaults, which are all quite reasonable if you're using both Apollo Server and an Apollo Client together. In this module, we'll start by learning what



normalizing data for caching is. It essentially means assigning a primary key to records. It's really important for making cache efficient. We'll then learn how to read and write data to the Apollo cache, though much of it gets set automatically, and as an example, we'll add a sort button to our speaker list, which effectively just sorts the cache, and that updates the React UI. In the last module, you saw how the Apollo cache automatically integrated with our speaker updates, so that after our speaker updated, no new data needed to be downloaded. In this module, you'll learn the custom logic necessary to do the same for inserts and deletes. Finally, you'll learn how to implement an optimistic UI by coding the Favorite and Delete buttons on our speaker page to instantly reflect the user action, even though the server has yet to respond. In case you came directly to this module without watching earlier ones, you'll need to make sure that you have the right source code loaded on your computer. Make sure you have the two folders on your local computer from the GitHub repo in module 4. Those folders are ApolloServer and ApolloClientReactApp. We won't be modifying our Apollo Server, so once you start that server, you can leave it running for the rest of this module. To launch it, switch to the ApolloServer folder and run `npm install` and `npm start`. Similarly, for the Apollo Client, switch to the ApolloClientReactApp folder, run `npm install`, and launch the development server for the web app, `npm run dev`. Let's get started.

Normalizing Data for the Apollo Cache

I find the best way to explain how Apollo normalizes data for its cache is to explain it with an example. Let's start out by first looking at a GraphQL query that we are sending to our server, then looking at the results the server is sending back, then looking at the actual cache that is stored in our Apollo Client. Here we are in our JavaScript code with our `GET_SPEAKERS` query defined using `gql`. Notice it requests the `speakers` object, which has one object under it, `datalist`. `datalist` has four fields under it, `id`, `first`, `last`, and `favorite`. Switching to a running app that is the result of the query execution, let's bring up the Chrome debugger and navigate to the Apollo tab. In the Queries tab we have our queries sent. Let's click on the last one. It's almost but not quite the same as what we sent. Notice our Apollo Client added a `__typename` field to both the `speakers` object itself and each individual `datalist` item. Now, let's run the query in our debugger, and that will show us not only the query itself but the downloaded results. Notice that for each speaker or item on the `datalist` there is an additional field, `__typename`, which is assigned `Speaker`. Notice that the speaker object, that's the top level of the query, has a new field `__typename` set to `SpeakerResults`. Let's now click down to the Cache tab, and let's see what the Apollo Client generated from that and inserted into our cache. Notice we have six entries. What Apollo did was to split out each speaker into its own separate item. That's what these five items are. Then it created one entry named `ROOT_QUERY` that's essentially the top level of our query. What the Apollo cache



does is it always tries to break down big lists into their individual parts and store each part as a separate cache entry. It has to know a primary key for each part, which, in our case, it assumes is the id. Remember, back in our Apollo Server schema definition in an earlier module we defined the id as the type ID!, which tells Apollo this is our primary key. Looking at each of the individual cache items, notice it's essentially indexed or keyed by the `_typename` that was downloaded as part of the queried results, concatenated with the speaker's primary key, id. Coming up later when we talk about data resolvers and we add a session object, you'll see that not only do we have `speaker:number` cache entries, but also `session:number` entries also. That's basically all there is to Apollo cache normalization. The cache is just a flat list where the individual objects, each speaker in our case, is keyed by the `_typename`, which is speaker, and the speaker id.

The Pattern for Adding, Updating, and Deleting from Cached Lists

Updating our Apollo cache is an indirect way of updating what React holds in state in a component. That is, typically, we are displaying GraphQL retrieved data from what was returned from the `useQuery` React hook. Since that return is React state, any change to it gets reflected in our UI. Behind the scenes, that data is tied to the Apollo cache, which means that updating Apollo cache updates our React component state, and that updates the rendered DOM in the browser. Remember from our last module, we tied the click event of our favorite icon to the toggle speaker favorite mutation, so clicking that not only updated the server with the change, but also updated the cache. Keep your eye on the cache data favorite field, and as I click on the icon, it toggles between true and false. What we did not talk about in the last module is what happens when we insert or delete a new record. If you remember, in both our insert and delete mutations after the insert or delete executed, which essentially means the server got sent the change, the only reason our client updated is because we included in our mutation the option `refetchQueries`, which made the Apollo client re-request `ALL_SPEAKERS` again from the server, and since its server data changed, the correct new speaker list is displayed. `useQuery` essentially just completely refreshed the Apollo cache and replaced everything on the screen with the latest data from the server. Because React is efficient at updating the DOM, that is, React makes the DOM diff calculation to figure out what to update, the screen does not change, but behind the scenes, both our React client and the React server were busy. The point of all this is so we can talk about what we have defined here as our `ROOT_QUERY` in cache. That list of speakers, remember, in our cache and linked to the UI, is what is reflected by our React app. If we were to surgically remove one of those items from our list, it would stop showing on our UI. If we were to surgically insert a new item on the list, then we'd see a new speaker on the page. Turns out we can do exactly that, and we will coming up in



the next clip. When we do that, we'll no longer need the `refetchQueries` on our mutation, as we're taking care of the UI updates ourselves. No extra work on the client to update all the cache based on a fresh download, and, of course, the server didn't have the extra work and bandwidth to worry about; a win for both client and server.

Fixing Insert and Delete Mutations to Modify Apollo Cache

Previously, we built the React app that utilizes the Apollo Client for get and update operations. We've got an insert button in the app toolbar for adding these speakers and clickable icons, favorite and delete, for updating and deleting speakers. Of these three operations, only the updating one, toggle the speaker favorite status, is optimized for use with cache, and that's because its behavior is built into the Apollo library. In this clip, we'll go on to optimize cache use for both insert and delete as well. You'll also learn all about what that optimization actually does. Here's our `index.js` of our React app, which represents our speakers page. At the top, are the GraphQL queries and mutations. `GET_SPEAKERS` is the query that gets our speaker data, and the other three mutations, `AddSpeaker`, `ToggleSpeakerFavorite` and `DeleteSpeaker` do the updates. Let's first look at our insert operation that executes based on the insert speaker button in the toolbar being clicked. It calls `addSpeaker`, which is returned from the `useMutation` call above. The mutation itself modified the data on the server, but remember, I said earlier, inserts and deletes don't modify the Apollo cache, meaning that if we didn't have the `refetchQueries` force a refresh of all the speakers by executing the `GET_SPEAKERS` query again, the new speaker would not be rendered to the UI. What we really want is our Apollo cache to update without having to have the `GET_SPEAKERS` query called, and hence, no extra round trip from our Apollo client to our Apollo server. If we do update our cache, our React UI will automatically reflect that, because well, that's what the Apollo Client library does for us. It essentially, behind the scenes, links our cache to the Apollo state for the component that's being rendered. So how do we do that? Instead of assigning the property, `refetchQueries`, passed into our `addSpeakers` mutation method, assign the property update to a function instead. Now, when the mutation completes, instead of re-querying our server with `GET_SPEAKERS`, instead, the method assigned to update is executed. The method takes two incoming parameters. The first is the Apollo cache object, and the second contains the results of the mutation. Those results include the properties `data`, `loading`, `errored`, `called` and `client`. `Data` is what we're going to focus on, and it's what is returned from, in our case, the `addSpeaker` mutation. Let's take a look back at what it looked like when we executed this `addSpeaker` mutation in our GraphQL playground. Notice what's returned is simply assigned to the property `data`. This is exactly what is returned to us in the mutation result. To get the inserted record itself, we just need to de-structure `addSpeaker` from that data.



Back to our code. The de-structuring to get the new speaker record is just data: (addSpeaker), and that gives us addSpeaker as our new added record. So what does it take to modify the cache to reflect our new speaker record? Basically, just two steps. First, get the cache in the function, and then update it. To get the cache, we execute the function `cache.readQuery`, passing in our `GET_SPEAKERS` query, and de-structure the results of that to get just our speakers data. This method only reads from cache, so you don't need to worry about it ever calling our GraphQL server directly. Then, writing the data back to cache is easy. Call `cache.writeQuery`. Pass in the query whose cache we want to update, then set the data property to what we want our new cache data value to look like, that is, looking back at our Apollo debugger, the root query shown is the results of what that `getSpeakers` use query had called, and we want to replace the entire speakers item. Just like the original query, we create a speakers object. Because it's going into cache, we need to give it a `_typename`. That's actually `SpeakerResults` in this case, and then set the property `datalist` with a new list of speakers. Remember, we are not updating cache, we're replacing it. We create an array. First entry will be our new speaker. Then, follow that with all the original speakers in the cache using the spread operator. Notice, we added the speaker to the beginning of the list, so it will show at the top. We could just as easily have added it to be shown at the bottom by putting it after the `speakers.datalist`, or even inserted it in some sort order, as might make sense for our app. Deleting a speaker record is similar. Let's copy this code down to where we have our delete mutation, replace `refetchQueries` with our update. Then, removing the speaker record from cache is almost identical to inserting. First, change the de-structured result from `addSpeaker` to `deleteSpeaker`, as that's what gets returned from our speaker delete mutation. Then, instead of assigning a new array with a record inserted to `datalist`, change that to a filter that returns a new array after removing the appropriate record. That's it. Let's run our app. Add a new speaker. That works. Notice the speaker is added to the top. Then, let's delete a speaker by clicking the trash icon. Sure enough, that disappears. Success. Next, Optimistic UI. We'll get rid of the delay that happened when we clicked on the trash icon, and we'll also get rid of the delay that happened when we clicked on a speaker favorite icon.

Implementing Optimistic UI for Updating and Deleting Speaker Records

The Apollo GraphQL client makes it unbelievably easy to implement an Optimistic UI in your app. You'll see in this clip that it's literally just a handful of declarative lines of code and it works perfectly. In contrast, in my recently released course titled *Designing React Components* I dedicated a full module to basically doing the same thing in standard React code. It's the final module, and it's titled *Component Techniques Including Optimistic UI*. So what is an Optimistic UI? Think about Twitter and what happens when you click on the like



or heart icon. It immediately reacts even on a very slow internet connection. Twitter does this by immediately updating the UI when you click on the icon and not waiting for the server to respond with a success message before showing the new state. In this clip we'll do exactly the same thing for our speaker favorite icon. And, in addition, we'll implement that feature with a delete or trash icon such that the speaker row will be removed immediately and will not have to wait for the server to respond. It's called optimistic because it assumes a result. If the server call fails the UI needs to revert to the previous state so as not to show incorrectly. Let's code an Optimistic UI for toggling our speaker favorite. Let's start out in our `index.js`, that's our main speakers page, and navigate to the `onClick` event associated with the favorite speaker. We are executing the function `toggleSpeakerFavorite`, which, if you remember, is what we returned from our `useMutationReact` hook that does the toggling, basically updating the speaker item favorite property. It currently gets passed into a single object with just the property variable assigned, telling the mutation which speaker to toggle. For Optimistic UI, all we have to do is add another property, `optimisticResponse`. What we assign here will essentially be returned immediately when `toggleSpeakerFavorite` is called so that the UI will update to the new return state without waiting for the actual result. Looking at our GraphQL playground from our first module and looking at what `toggleSpeakerFavorite` returns, we want to return this exact same format in our `optimisticResponse`. Let's assign values to our `optimisticResponse`. First, the `_typename` that's `Mutation`, then our `optimisticResponse` itself. `toggleSpeakerFavorite` as the object name and then using ECMAScript shorthand syntax for properties `id`, `first`, and `last`. And then set `favorite` to `!favorite` to affect the toggle. We also need to add the `_typename` `Speaker` as this is what Apollo uses to normalize the cache along with the `id` field. Let's copy this code and scroll down a little and do the exact same thing for the `deleteSpeaker` click event. Add the `optimisticResponse` object, assign a `typename` `mutation`, then the object `deleteSpeaker` with `id`, `first`, `last`, and `favorite`, followed by the `_typename` `Speaker`. That's it. We've got our Optimistic UI now set with the `optimisticResponse` attribute passed into the `useMutation` for both `toggleSpeakerFavorite` and `deleteSpeaker`. Now, running our app code in the browser, clicking on a Favorite icon for a speaker, and notice it immediately updates after I click it. Deleting a record by clicking on the trash icon for a speaker, immediately the speaker goes away. Just like Twitter, this is a great user experience. Next up, we'll add a Sort button to our toolbar and show how clicking that button sorts the Apollo cache. And that's reflected in the speakers in the browser also reordering.

Sorting Displayed Speakers Using Apollo Cache

Based on everything we've talked about in this module, it's a very small stretch to think about sorting speakers from a click event. Let's code it. Start out by adding



another button to our toolbar component and title it Sort Speakers by Id Descending. Add an onClick event to it and let's call a made up function name, and we'll just call that function `sortByIdDescending`. Let's assign that function to our passed in properties so we can execute it on our main speakers page where we currently have all our mutations loaded. Switching back to the main Speakers page, `index.js`, let's add to the invocation of the toolbar another function as an attribute, `sortByIdDescending`. Just like for the `insertSpeakerProperty`, let's assign a function to that property that will get executed when the browser user clicks on the toolbar sort button. What are the steps to do this sort? First, get a reference to the Apollo cache, then read the cache, sort the speakers list in that cache, and then write that list back to the cache. Here is the only non-obvious part that we've not covered yet. How do we get a reference to the Apollo cache? In our mutation update event, the cache was passed into our function as a passed in parameter. In this case, we are defining our own function that Apollo does not know about. Luckily, the Apollo team has given us a custom React hook for that included in the library, just add to our named import from Apollo client, `useApolloClient`, then just like we call `useMutation`, call `useApolloClient`, and it returns us one value Apollo client. Now in the sort button event, we can read our cache by calling `apolloClient.cache.readQuery` passing in our `GET_SPEAKERS` query, then return the destructured value `speakers`. Writing is just `apolloClient.cache.writeQuery` passing in first the query again, `GET_SPEAKERS`, and then just like we updated our speakers data in our mutation update method, we pass in our new data. That's our data object with the speakers property, and that gets our `_typename` `SpeakerResults` and now we need to assign to `datalist` a new speaker array that's sorted. Using the spread operator, we just list the speakers, then apply a simple sort method that reverse sorts by id. That's it. Running our app again, paying attention to the speaker IDs here, press the Sort button and immediately the speakers sort in reverse ID order. That does it for our deep dive into the React client cache. The big takeaway that I hope you remember here is that cache is intricately tied into the React component state and that any client-side modifications of cache are immediately, and with great responsiveness to the browser user, reflected in the UI. We showed multiple examples of how we can take advantage of this cache state relationship, first, by fixing our speakers update and delete mutations to update cache directly instead of having to refetch speakers from the GraphQL server. Then we implemented an optimistic UI using the Apollo clients built-in functionality available from the React custom hook `useMutation`. Finally, we implemented sorting speakers by just reordering what is stored in the Apollo client cache. In the next module, we'll take the Apollo client one step further integrating local state management with and without using the Apollo client cache. It turns out not only is Apollo excellent at implementing a GraphQL client, but also, it's a great state manager in a similar way to how you'd use Redux or other state management tools.



Using Apollo Client for Redux-like State Management

Understand How to Manage State With the Apollo Client

Hi. I'm Peter Kellner, and welcome back to my course covering Apollo performance and to this module on state management using the Apollo Client. The Apollo team has done an amazing job of integrating state management into their client library. By leveraging some of the same techniques you've learned already with GraphQL, you'll be managing local state in your app in no time. To manage state, there are three Apollo Client features you'll need to use. Let me outline them, explain how they fit together, and then for the rest of this module you'll learn step by step how to bring them together to manage global state in a React app. First is learning how to create virtual or, as Apollo calls them, local-only fields that can be added to an existing schema type, like our Apollo speaker type for example. Then you'll learn about creating and updating reactive variables, which can be thought of as global variables that have the ability to influence results of all GraphQL queries. Finally, you'll learn to integrate local-only fields and reactive variables so that your app, with just standard GraphQL queries, can participate in full state management. Let me explain in a little more detail what I mean using pictures. Think of local-only fields as schema fields that are not stored in your GraphQL server, but are created or calculated locally in your GraphQL client. The value of these local fields can be anything you want, including being based on other fields returned from the GraphQL server or any other source. Adding to that, let's make these local-only fields also be able to be dependent on reactive variables. Think of reactive variables as just global JavaScript variables that are created off of an Apollo instance created in your React app. These reactive variables will be exported such that not only will you be able to read them from anywhere in your React app, you'll also be able to update them. Let's assume we've got a bunch of GraphQL queries we've written that have different combinations of real fields that you get from your GraphQL server, as well as local-only ones that we've somehow defined associated with these queries. Next, assume that you have your React app built and you currently have assigned these queries with `useQuery` in your components and that the returned data from `useQuery`, remember that's real React state, is assigned to some UI components like textboxes and drop-down menus. That means if you update the Apollo cache, like we did in the previous module using mutations, those fields will automatically react and get the new updated data from those mutations. Where this all leads is say you have a textbox showing data from a local-only field that is set to show state that was retrieved from a GraphQL query. That query included that local-only field. Now, let's say from a button we call one of our React variables, `setter`, one that's inside our local-only field, and that reactive variable set causes



the state to update. And that's because our local-only field was included in the query that we retrieved that text value with. And that value change causes our UI to update. Okay, I get that what I just explained is a lot to understand and process. Throughout this module we'll give several examples of combining reactive variables and local-only fields, which will make it a lot more clear.

Refactor and Reorganize Our React Client Code

As we've added more features to our client-side React app using Apollo, we've not been doing appropriate refactoring along the way, that is mostly in our home component, `index.js` and our toolbar component `toolbar.js`, we've let them grow a little bit too much. It's helped keeping things simple while explaining, but not good for the long run. Let's fix that now and extract components where it makes sense. Starting out in our `index.js`, let's extract our GraphQL query and the three mutations. Let's put them in a new folder, GraphQL and name them `queries.js` and `mutations.js`. Notice each query and mutation has its own named Export. I have switched now to the new project, so going forward, this is the refactored code that we're looking at. Back in `index.js`, let's replace everything there with a new component, `SpeakerListItems` that we'll put in the `source/components` folder. Navigating there, we've got our `GET_SPEAKERS` GraphQL query referenced and we've simplified our render to now include our `Toolbar` component on top and a new `SpeakerItem` component for each speaker. That `SpeakerItem` now has all the functionality self contained for favoring and deleting speakers as those actions are only available from the speaker component itself. Similarly, in our extracted toolbar, we have the `addSpeaker` GraphQL query referenced, and since the `Insert` button is on the toolbar, that functionality is now in this component. Lastly, from our `_app.js` file, we've extracted a custom hook and named it `useApollo` so that our config for Apollo is maintained in our GraphQL folder as `apolloClient.js`. I suggest that if you've been coding along you either refactor yourself slowly as that's good practice for your React coding or just pick the code up in the GitHub repo, it's in the folder module 2 clip02-Organizing-and-Refactor-Code. There is absolutely no functionality change in this code versus what we finished the last module with.

Field Policy Overview and Creating a New Combined First-last Local-only Field

The Apollo 3 GraphQL client supports creating virtual, or as Apollo calls them, local-only fields. This is our server based schema for our GraphQL type `Speaker`. Say we want to create a local only field named `fullName` that we reference as part of the GraphQL query we sent to the server. We want it to return first and last concatenated. When we send our query, we don't want this



fullName field to be passed through to our real GraphQL Server. We do, though, want our Apollo client to return it to us after doing some local computations on it. We're here back in our React app that includes the Apollo client and we're looking at the actual query we send to GET_SPEAKERS, it matches exactly what's in our GraphQL server. Let's add a new field which we want to be local only, fullName. We do need to tell our Apollo client that this is not something the server knows about, so we add the directive add client to it. It's going to be our Apollo client's job to both compute this fullName local only field and not send it on to our server. Let's now add to our Apollo client code what's necessary for this field to be valid, as well as the code to process what it returns. The way we do this is create what Apollo calls a type policy and, more specifically, a cache policy. All type policies, including cache policies, of course, are declaratively instantiated when the cache itself is created. Moving to our Apollo client custom React hook, that's where we create our Apollo client instance. Let's focus on the creation of our cache, that is what happens when we call new InMemoryCache. At the moment, we are not passing any arguments to our cache provider. Let's change that, but first, creating a new options object, empty at the moment, then passing it as our only parameter to in-memory cache. Our config can contain multiple properties, for now, we're going to add one called typePolicies, which is what we use to create local only fields in our schema. TypePolicies are by schema type so we add Speaker, and under that, a list of local only fields we want to create. For now, we just want one, fullName. To fullName, we add a read property which gets assigned a function that our Apollo client will call after its GraphQL server returns data. It turns out we can create a cache policy on an existing field, and if we do that, the first parameter is the value returned from the server and we can use that value in our function or we can default it if there is not one already. For now, we'll just ignore it with an underscore. The second parameter is an object that Apollo uses to pass in what they call modifier functions. The particular function we want to use here is readField, which gives us access to other fields of the current speaker query. All that's left is to return a string that concatenates first and last together. Using the readField modifier function, let's do that. Notice that I've put ... between the names so we can be confident that our type policy is actually working. Let's now update our app to use this fullName field, instead of first last like it was using before. Going back to our speakers list items component, we can assume now we are getting it back from our use query call that access the query we just modified. So let's add it to what we passed to our SpeakerItem component, that is, in both the map and what we passed to the SpeakerItem component itself. Now, in the SpeakerItem, add to our list of receiving props fullName. Then in our render, let's replace are two separate fields in the h4 to just one, fullName. Finally we can see if our efforts worked. Browse to a running app. Sure enough, here it is, including the ... Next up, we'll look at reactive variables and how that integrates with a custom theme on our page.



Working with Reactive Variables Including Light/Dark Theme Example

Reactive variables are just like any other JavaScript variable, with the exception that the running application can observe and react to any changes in them. What does that mean exactly? Let me explain by building into our conference web app the functionality to toggle the theme for our app from light to dark or dark to light. And you'll see how easy they are to use. To start out, we need to create one of these reactive variables. Let's do that in our ApolloClient class. That's where we instantiate our React client because it's included from our Next.js app in `pages/_app.js`. The method Apollo gives us for creating React variables is `makeVar`, so let's include that in our import. Right below that let's use it, `const currentThemeVar = makeVar`. And let's default it to dark. To make it available to other components, let's export it so that they can include it as a named import. Switching to our speaker's list item component, let's use it. Our plan is to add a toggle in our toolbar to switch our theme between light and dark. That means when we use our current theme value it needs to be in state so that the display will rerender when it changes. That means we need to use the Apollo custom hook, `useReactiveVar`, to get that state. Add that as an import at the top. Then, like when using all hooks, include it in the function definition below, `const currentTheme = useReactiveVar(currentThemeVar)`. `useReactiveVar` acts a lot like `useState` in that it returns state, which is reactive. Now we need to import our current theme var, `import {currentThemeVar} from '../graphql/apolloClient'`. That's where we just created it. Now we can use it to set our theme appropriately. We happen to have the CSS class `dark` that if we enclose our rendered speaker list in it will display on a dark background. Adding a JavaScript expression to set the class, that is update the class name attribute from just `fav-list` to if `currentTheme` is dark set it to `fav-list dark`. Otherwise just `fav-list`, which is our light theme. Running the app, we've got our dark theme working. Now for the good part, let's add the theme toggle to our toolbar. Moving to our Toolbar component let's first update our imports to what we know we'll need. Add the hook, `useReactiveVar`. And below that, `import {currentThemeVar} from '../graphql/apolloClient'`. Next, get the current value of our theme so we can display it as a value in the drop-down list. That will display in a moment after scrolling down. That's `const currentTheme = useReactiveVar(currentThemeVar)`, just like we did in our speakers list component. All that's left is to add the drop-down toggle so that the browser user can change the theme. Adding a list item to the toolbar, name it `Theme`, then create a simple select list. Surround that with a label attribute and assign it some helpful CSS classes. Make the default value the `currentTheme`. Remember, we got that from our `useReactiveVar` hook. Then let's program the `onChange` event. What we need to do is call our reactive variable with a single parameter to change it. Basically, the syntax for using reactive variables is if you want the value directly just execute the reactive variable itself with no parameters. If you want to set it, call it with the parameter you want to set



it to. That said, let's run our app. Comes up dark, showing in the drop-down dark is the theme. Toggling it to light, sure enough the actual theme toggles and our drop-down shows the current value. The takeaway here is that all we had to do is set our reactive variable to a new value. And because we called the hook `useReactiveVar` we had access to that variable as state, which of course changed when we changed the reactive variable's value. You may be thinking to yourself, this seems a lot like using the React Context API. Well, you'd be exactly right. When using either the React Context or the GraphQL Apollo Client there needs to be a provider that wraps the component you are interested in using. In this example, you can see how similar they look for just a generic implementation. Bottom line, if you're going to use the Apollo Client you might as well use reactor variables because you've already done all the work to include the Apollo context in your app. Coming up next, we'll enhance our app to include a checkbox next to each speaker that is completely controlled with the local React state combining reactive vars and local-only fields from our schema.

Combining Reactive Variables and Local-only Fields for Redux-like Behavior

At the beginning of this module, our goal was for you to understand how Reactive variables can influence local-only fields, and do it in a way that flows to the correct state variables in your app. In other words, being able to read and write Reactive variables, and at the same time, have all UI components that are dependent on those Reactive variables, update correctly. Let's now bring it all together by extending our conference app's speakers list to include a check box next to each speaker that's only managed locally in the React client app. The GraphQL server knows nothing about these check box fields. Starting in our speaker query that we used to list all speakers, let's add a new field, `checkBoxColumn`, with the directive `@client`, making it local-only. Over to our type policies in our app Apollo client file, below full name, add a new field, `checkBoxColumn`, with a read function exactly the same as our full name field defined earlier. For now, let's get the value of the current row's id. Then, if it's odd or even, return true or false. That will give us something to test our UI with. Switching to our speaker list items component, that's where we have our use query, that processes error GraphQL query, add the `checkBoxColumn` to our other columns we passed through to our `SpeakerItem` component. That is, `checkBoxColumn` goes as a parameter of the map to a property of the speaker item. Now, to the speaker item component. Add the new `checkBoxColumn` as a receiving prop. Scroll down to a render section. Let's add a check box as the first column. My preference is to use the Font Awesome classes for images. Create a span, then assign to that class name, either `fa-check-square`, or `fa-square`, depending on the check box column's value. Let's browse now to our app. We have our new column, and it's checked differently, depending on whether the id is odd or even. We need to track the state of each check box locally. Let's do it by



creating an integer array of all speakers that are checked, then assign that integer array to a Reactive variable. Going to our Apollo client, let's add it there below, where we created our current theme bar. Export `const checkBoxListVar = makeVar([])`, initialized to an empty array. No speakers checked to start. Now let's go down to our type policy for our check box column and replace our odd even logic. First, get our selected speaker ids list from a Reactive variable, then return either true or false, depending on whether the current speaker id is in the selected speaker ids array. All that's left is to program the click event on the check box and make sure that the integer array accurately reflects the state of that check box. We're back in our `SpeakerItem` class, which is where we want to add that on-click event for each speaker record. Let's start out by getting the list of currently checked speakers. We get that by getting our speaker list as a Reactive variable. That's `const selectedSpeakerIds = useReactiveVar(checkBoxListVar)`, and of course, we have to import both `useReactiveVar` and `checkBoxListVar`. Scroll down to our checklist span, and add a click event to it. Have it replace our speaker ids list by calling `checkBoxListVar` and passing it the new list. That's how we set Reactive variables, adding the logic, if the current column check status is true, then, return a new list with the current speaker removed, and if false, add the speaker to the list using the spread operator and id of the current speaker. Remember, like state updates, we're replacing the list, not updating it. So that's pretty much all our functionality. We now have our ids list being maintained when the user clicks on check boxes, and we have our local-only fields rendering those check boxes. Let's now create a button on our toolbar that iterates through only the checked speakers and toggles those speakers' favorite status from true to false, or false to true. Back to our toolbar component, let's add a button titled `Toggle All Checked`, meaning for all speakers that have their check box selected, toggle their speaker favorite value. To achieve this, get a list of selected speakers by calling `checkBoxListVar`, with no parameters, then, iterate over that list with a `for each`, call the mutation, `toggleSpeakerFavorite` with a `speakerId` variable, and that's basically it. All that's left, is we need to make sure we have our imports included. Scrolling back to the top, we get our `toggleSpeakerFavorite` with `useMutation`, we import the mutation itself, and finally, the Reactive variable `checkBoxListVar`. That's it. Let's run our app. Let's select the top three speakers. Keep an eye on their favorite status. Click on the new toggle favorite button. They all toggled. Click it again. They all toggled back. Normally, about now, at the end of each module, I have a slide, and I review what I think the important takeaways are from that module. In this module, however, I don't think that's necessary. If you followed the last example we went through, that included using Reactive variables, local-only fields, and how they interact with state, that's everything. Looking again at our opening slide, hopefully it makes more sense now. Reactive variables are at the center. They can influence local-only fields in a GraphQL query, and that leads to the appropriate state updates that are dependent on both the Reactive variables themselves and the GraphQL query that depend on these local-only fields.



Learning to Use Offset and Infinite Pagination with Apollo

Pagination Basics

Hi. I'm Peter Kellner, and welcome back to my course covering Apollo performance management. This module is all about data paging using the Apollo GraphQL client and server. The reason we implement paging is because we have too much data to show in the browser at once. That could mean one of two things, it's either too hard to get all the data from the server because of server loader bandwidth considerations or it's too much information for the browser user to take in all at once. In other words, a UI problem. Pagination implementations typically fall into two categories. The first I think of as numbered pages. Typically that's good for things like books or conference speaker pages where the dataset does not change or change much over time. We'll implement numbered paging coming up in this module for our speakers list. The other type of paging is broadly referred to as cursor based. The idea is that every item being paged can be referred to with a discrete value that's typically opaque, meaning that it doesn't give you any useful information about the item itself. But it does give you a reference you can directly access it with. The reason we use cursor-based paging is that we often want to implement a strategy where we want to show a limited number of records directly after or before the current one. Say, for example, you're looking at the latest tweets and specifically the latest 10 of those. By the time you're ready to see the next 10 many more tweets may have come in. Basically, it's a way to keep going from where you were instead of just randomly jumping into a list based on some old, stale offset. Cursor-based paging can be either single or bidirectional. For example, again in Twitter, if you scroll down through your own feed new tweets will be dynamically loaded as you scroll down. Twitter is doing this by remembering the last tweet it downloaded for you and then continuing to load them as they come into view. What's also interesting is if you scroll back up to the top Twitter will also load new tweets that you've not seen by remembering the first tweet it showed you when you first visited the page and then loading newer ones based on that. The implementations of cursor-based paging can get quite complex. In the GraphQL world the most common implementation is based on the relay spec and relies on what they call the GraphQL cursor connection specification. It requires your GraphQL server present the data in a very specific way and, along with each download, provide metadata, including things like the first cursor shown, last cursor shown, is there a next page, is there a previous page, and other things. In this module we'll implement an example of just forward load more type cursor paging where you'll be able to click a button at the bottom of a short list of speakers and that button will concatenate speakers to the bottom of your existing list. Next up, we'll upgrade our GraphQL server to handle numbered paging pagination.



Updating Our GraphQL Server to Support Number Based Paging

We're in the Playground from the Apollo server we built in the first module of this course. It currently always returns all our speakers data and provides for no paging functionality. Our objective here is to provide to our GraphQL client enough capability so it can implement numbered paging. It requires that we add two parameters to our schema, offset and limit, to our speakers query, and also the total number of items returned unpaged. The code we will work on now will be from the end of the previous module. That's in the GitHub repo module 05 clip05. In this clip, we'll just be working from the ApolloServer folder. As with all our clips that we code, the final code for this clip will be in module 06 clip02, also in the Apollo Server folder. Let's make our changes to support number-based paging. Let's first update our schema to reflect what we need in the fileserver.js. Starting with our speakers query, let's add two parameters, offset, which is how many speakers to skip before returning, will default out to 0, and limit, which is actually how many to return. Defaulting to -1 means returning until the end. Then let's update our SpeakerResults to include a new type, PageInfo. Let's create that type and just give it one field, totalItemCount, which is how many speakers are in total. We'll need to return that on every speakers query. All that's left is to update our resolver, and specifically the query speakers resolver. Let's start out by destructuring our passed-in args, offset and limit. Then let's set our return datalist to a filtered list of speakers based on our offset and limit. Here's a simple, safe calculation that does the extraction. We could use slice, but for the small number of speaker records, thousands at most, this is plenty fast. We also need to add to our return our pageInfo. That's just our total count of all speakers, not just the ones returned. Running with npm start, we can paste into the Playground a query that returns just three records with an offset, and it works. Next up, we'll build this functionality into our React app with the Apollo Client.

Numbered Page Pagination Including Forward and Back Navigation

We've got our GraphQL server support now for our number-based pages working. Let's update our React Apollo Client app to support it. There is a bunch of UI stuff here, but it's not complicated and brings in much of what we learned in the previous modules, especially the last one covering Apollo Client and state management. Let's run the app before we add the paging and make a plan for how we're going to add it. To run our app, go to a terminal prompt first in the ApolloServer folder and issue the command npm start. Then back to our ApolloClientReactApp folder and give the command npm run dev. We've got about 10 speakers here, so let's think about paging them three at a time. Let's assume we're going to build a paging toolbar that shows the current page has a



first and previous button, as well as a next and last button. At any given time, the app is going to want to know four state values that it can reference from any component, limit, offset, currentPage, and totalItemCount. More details on that coming. Let's create a state object, also known as a reactive variable, that holds those four values, and let's call it paginationDataVar over to where we initialize our Apollo cache, ApolloClient.js, and let's establish that as an export, export const paginationDataVar = makeVar, and then pass in those properties. Set their defaults, limit is the number of speakers to show at a time, offset is how many speakers to skip before starting to show them, currentPage is, well, the current page, and totalItemCount is the total number of speakers, not just the one showing. Let's update our SpeakersListItems component. That's the one that executes the GraphQL query to get our speakers into state. First thing, let's grab our paginationData state we just created. At the top of our component, we declare const pagination data = useReactiveVar(paginationDataVar), and of course we have to import paginationDataVar from our apolloClient where we just created it. We'll do this in every component we want to get our update, our paginationDataVar. In the next clip, we'll completely build out our pagination toolbar based on this paginationData.

Add State Change Actions on Pagination Toolbar

Picking up where we left off in the last clip, let's continue building our pagination toolbar. As we're now planning on passing offset and limit using our GET_SPEAKERS query, we need to add to that query the passed-in parameters. Moving over to query.js, we add those variables so our GraphQL server knows about them and can respond accordingly. We also need to add our pageInfo to this query. Back to our SpeakersListItems component, let's start by destructuring limit and currentPage from our paginationData. Then to our GET_SPEAKERS query let's add the variables offset, which is just currentPage times limit, and limit. If we were to run our app, we'd just get three speakers back, starting at offset 0, but no way to navigate to other pages. Let's add that page navigation to our existing Toolbar. Let's pass in our total number of speakers, totalItemCount, navigate to our Toolbar. Let's do the same getting our paginationData as before, add paginationDataVar to our import from the apolloClient, and then call useReactiveVar to get our paginationData. Let's add totalItemCount as a receiving prop to our Toolbar. Scrolling down to the render section of our Toolbar, let's calculate the lastPage based on the totalItemCount and the pagination limit. Now add a new component that we've not written yet, pagingOffsetLimitControl, and pass in lastPage as a property. This will basically put our pagination control right in our Toolbar. Scrolling to the very top, let's import the function we haven't yet created, but we will in a second. So let's now create an empty PagingOffsetLimitControl function. As usual, we're going to want our paginationData, so import useReactiveVar from apolloClient and import paginationDataVar from graphql/apolloClient, then create our data, const



`paginationData = useReactiveVar(paginationDataVar)`. Destructure `currentPage` and add the receiving prop `lastPage`. Then we paste in just the HTML and class definitions for a four-button toolbar, First, Previous, Next, and Last, with some nice CSS classes. Then in the middle let's put our `currentPage` number from our `paginationData`, adding 1 because no one wants to see page 0. Now we have something to run. Browsing to our app after `npm run dev`, we're on page 1, and our first three speakers are showing, as well as our nicely formatted pagination toolbar. Click First, Previous, Next, and Last. Nothing works, because we've not coded any click events. Let's do that. First the Next button. Add an `onClick` event to the anchor tag, check to make sure we're not on the `lastPage`, then update our reactor variable, `paginationData`. Because this is an update to state, we can't just change it, we have to replace it, first spreading `paginationData`, then replacing `currentPage` with `currentPage + 1`. So here's the beauty of what we just did. Remember back on our `useQuery` where our variables passed in, `offset` and `limit` were dependent on what was returned from `useReactiveVar`, `limit`, and `currentPage`. Updating `currentPage` tells `useQuery` it needs to refresh the speakers that it retrieved using `offset` and `limit`. All we had to do is update our `paginationDataVar`, and everything reacted for us. Back to our paginated Toolbar. Previous is exactly the same as Next, but decrement `currentPage` and make sure we're not going back before page 0. First and Last are just setting the `currentPage` to 0 or `lastPage`. Printing our app now, paging forward, back, first, last, it all works. Notice if I visit a page I visited before, no loading state. That's because Apollo Client cache is working for us. One last thing, if you try some of the functionality we built earlier directly modifying cache, it won't work. The fixes are in the GitHub repo. Briefly, though, here they are. These fixes are for the `sortBy`, `insert`, and `delete`. They all involve cache writes, in that you have to put back in the cache exactly what you took out. Here's the update to `sortByIdDescending` Here's the update to the `insertSpeakerEvent`. And for `delete` that's in `SpeakerItem`. We have to first import our `paginationDataVar`, and call `useReactiveVar`. Scroll down, and update our `delete`. That's it for fixed-number paging. Coming up, we'll build out cursor-based paging and, specifically, adding a Load more button to retrieve more records.

Preparing for Offset and Cursor Based Pagination

In the beginning of this module, we talked about the two types of paging. We're done with offset-based paging. Time to move on to cursor-based. For our previous example, offset-based, we added two parameters to our query, `offset` and `limit`. This works great, unless they don't change over time. That is to say, if while you're paging records are inserted or deleted from previous pages, you will either miss records or see records multiple times when paging. Cursor pagination minimizes that problem by keeping track of the last item shown, and then when the user requests the next page, the server requests the record after the last one already shown. This requires that a value be stored with each item that



uniquely identifies that item, and that value we will call the cursor. For cursor pagination to work, we need to add more fields to our pageInfo, specifically lastCursor, which is the cursor of the last item on the current page downloaded, and also a Boolean, hasNextPage, so the UI knows whether to show the next page navigation control if one exists. Remember, now that we are not using the offset-based paging, we have no way to know whether we're at the end of the list. To make it more clear, here is our new query, which we'll add to our GraphQL server momentarily, named speakersConcat. Notice the lastCursor has the same value as the last item from the query, and hasNextPage is true, meaning there are more pages to come. The way to query the next page is to pass in the lastCursor to the afterCursor parameter of the query. Then, no matter how many records were deleted or added before this, we'll get the next record. This isn't perfect, but it's much better than what we had before with numbered pages if the data is changing frequently. Next up, we'll upgrade our GraphQL server to process our new cursor type query.

Adding Results with Cursors to Our GraphQL Server

We're back in the folder that holds our Apollo GraphQL server. Let's create a new query called speakersConcat that we'll use for our cursor based speaker retrievals. Pretty much the entire server is in the fileserver.js that we've got here. Let's start out by updating our schema. To speaker, add the field cursor as a String, to our pageInfo type, let's add lastCursor as a string, and hasNextPage as a Boolean. Finally, to our schema route query type, add speakersConcat with two parameters, limit, which is the same as before, meaning the maximum number of items to download, and afterCursor, which will retrieve items only after a given cursor value. This does imply that our speakers data is always going to be returned to us in the same order. All that's left is to implement a resolver for a new query, speakersConcat. Like before, get all our speakers data from the REST service, but this time let's sort it by speaker's last name right up front. Just a side note, if you do implement sorting in your apps, it's always important to sort right after you retrieve from your external data source and before you do any paging calculations. Following that, destructure limit and after cursor from our past in args variable, and here's what's new. Let's assume a function named getOffsetCustom exists that we can pass in as parameters or retrieve speaker data and a cursor value to find. In this case, that value is the one passed in as an argument. Our new function will return the offset of that cursor value in our data. Scrolling up a little, let's create that function. You're probably wondering where the cursor is going to come from. Since we don't have one in our REST data, let's calculate it as a base64 encoding of our IT field to guarantee its uniqueness. Here's a simple function, getCursor, that does that for us. Now let's create our getOffsetCustom function, it takes in data and a cursor as a string. Using the array function, findIndex, we can get that offset we want returned. A quick check, if offsetBasedOnFind is -1, then return 0 indicating we want to start



at the beginning of the dataset or just return the actual found index. Now that we have our offset calculated, let's scroll down and use it just like we did in the speaker's query above, that is executing the same filter function. One more thing, though, while we're here, let's create our cursor value on each row by executing map on our return list and just add the property dynamically using getCursor. Next, let's create a pageInfo to return and include our totalItemsCount just like before, then create our two new properties, lastCursor and hasNextPage. Here, we have the simple calculations for those two new properties. Finally, we return, just like from our speakers query, dataList and pageInfo. Starting our server with npm start, browsing to Port 4000, we've got our GraphQL playground. Let's first paste in a query with afterCursor set to an empty string and limit set to 3, that gives us our first three speakers. Pasting the lastCursor returned into our query parameters, executing the query again, and we get the next three speakers. That's basically it for updating our GraphQL server to support speakersConcat as a query.

Updating Our Cache Field Policy to Support Load-more Style Pagination

Let's take a look ahead at our running React app that will have completed by the end of this module. After the page loads the first time, notice we have our first four speakers showing in the browser. Open our Chrome DevTools, browsing to the Apollo tab, then navigate to Cache. Our ROOT_QUERY has four speaker items in it. Back to the UI, press the Show More button. We get another four speakers added to our existing four, now showing eight total. Looking at our cache again, specifically ROOT_QUERY and our speakersConcat, we now have eight speakers in cache. And of course that's why eight speakers are showing above. So how did that happen? It happened primarily because of this code that you've mostly seen back in our Building a Simple and Performant Client App module. When we first loaded our page the first four speaker items were retrieved because we specified an empty string with our calling parameter afterCursor. This is very similar to what happened when we specified an offset of 0 with our offset or numbered pagination. What's different here is we've retrieved another variable, fetchMore, from our useQuery hook. fetchMore is a function that will call from our Show More button's onClick event. And to it we'll pass the variable afterCursor set to the last of our shown data, in this case the fourth speaker. When fetchMore is called we don't want to replace the return data with the four new items, but instead concatenate the four new items to the original ones showing on the screen. That means updating our Apollo cache from holding four items to holding eight. The design of the Apollo Client gives us the perfect place to do this. We can intercept the point at which Apollo is updating the cache. Apollo passes to our interception the existing data in the cache along with the new incoming data and gives us the opportunity to replace the complete cache with whatever we want. The way to do this interception of Apollo cache being updated is to add



a cache-type policy to our config object passed in to our InMemoryCache provider we instantiated back in the Apollo Client, which is in our apolloClient.js file. Previously we created two speaker-type policies, fullName, which basically returns first and last concatenated, and a virtual checkBoxColumn. This time we want to enhance an existing field, and that's our speakersConcat query. To our typePolicies we add a new type, Query, and to that add the fields property and the specific query we want to intercept, that's speakersConcat. Like before, we intercept the read of the field. Read is simple because we don't want to change our existing cache passed in. We just take the first parameter, which is the existing cache, and return it. But now we also want to intercept the write back to the cache, and one method Apollo gives us to do that is merge. The first two arguments merge gets are existing and incoming. That is what's currently existing in cache and what's incoming from the most recent query. Remember, the first time useQuery is called existing is empty and incoming has four new parameters. When fetchMore is called, existing has the four original speakers and incoming has the new ones retrieved from our most recent fetchMore call. Looking back at our cache for our speakersConcat field we're updating, notice it has typename, datalist, and pageInfo properties. Those are exactly what we need to return in the same format to update the cache properly. Let's do that, also taking into account what happens when our existing cache is empty, that is when the page first renders. That simply becomes return when existing is undefined, a new object that has the same _typename as passed in, then spread the existing datalist array into a new datalist, then spread the properties of pageInfo into a new pageInfo object. If it is existing, do exactly the same thing, but this time assign it to the datalist first to spread of the existing datalist and concatenate to that a spread of the incoming datalist. That's it for merge. Finally, we almost always want to set the property keyArgs to false to make sure that we have full control of the pass to an args to our field definitions. That's the plumbing of how we get our speakersConcat query to handle our show more cursor-based pagination functionality. Next up we will update our React UI to use this feature, which includes creating the new useQuery call shown earlier that included a new query definition, as well as the fetchMore function.

Updating Our React UI for Load-more Style Pagination

Previously we wrote a type definition for speakersConcat. Let's now create a new web page that uses that query to implement show more type pagination of data. First thing, let's add a new query to our app GET_SPEAKERS_CONCAT. It's almost identical to our existing speakers query in queries.js. So let's make a copy of that, change the name to GET_SPEAKERS_CONCAT, change one of the passed in parameters from offset to afterCursor, add to our speaker the field cursor, and finally to our pageInfo add lastCursor and hasNextPage. Next, let's create a new landing page to demonstrate our new show more cursor based pagination and



call it `SpeakersCursor.js` and put it in our `pages` folder. Let's have that file just simply render a new component we'll call `SpeakersLoadMore`, and we'll put that in our `components` folder. Let's build that out now starting from an empty component. First, import `React`. Then, import `useQuery` and `NetworkStatus` from our `Apollo Client` library. Finally, import our new `GET_SPEAKERS_CONCAT` query we just created. On our component's execution, let's call `useQuery` like we have previously. That is, we get back our state's loading, error, and data. And because we are doing this cursor based and this is our first query we pass in `afterCursor` as an empty string and make our limit 4 for now. What's new for our `loadMore` pagination is we want to get back from `useQuery` `fetchMore`, which we'll call from our `Show More` button to get more speakers. We also want to get back `networkStatus`, which gives us more fine-grain control about what's happening. Now we want to know the difference between the page rendering for the first time and when we just call `fetchMore` from the button. Let's not forget we also need to add the property `notifyOnNetworkStatusChange` so we get those notifications. We create a convenience variable for that, `loadingMoreSpeakers`. Then we can go onto checking our status variables. Render our loading message when we are not in a `fetchMore` state. Check for error. Destructure our data list, that's our speakers array, our `hasNextPage` Boolean, and the `lastCursor` value from our original query. Now finally, we can return, rendering our speakers list. That's the standard JavaScript expression `datalist`, and then map out or iterate through the speakers. Now for the good part, let's render our `Show More` button if `hasNextPage` is true. Let's add an `onClick` event and have it call the `fetchMore`, just passing in our last cursor we are currently displaying. Remember, we got that from the page info above. Instead of `Show More` as our text all the time, let's conditionally show either `Loading` as our button text or `Show More`, depending on our `loadingMoreSpeakers` value. Again, we set that above based on our network status returned from `useQuery`. That's it for our UI. Let's run it with `npm run dev`. Page comes up with four speakers and a `Show More` button. Pressing it, you briefly see the loading state. Then `Show More` again. Pressing it again and the last two speakers come up. And no more button. Works exactly as expected. Coming up, we'll generalize this show more paging strategy and apply it to a completely different dataset that is sessions data. First though, we need to add sessions data to our GraphQL server.

Adding Sessions Data to Our GraphQL Server

Let's add to our GraphQL server, conference sessions, in addition to conference speakers. We'll need to add new data to retrieve, so I'm updating the `db.json` file that our JSON server uses to include both sessions and also a join table, `sessionSpeakers`. That way, we know which sessions belong to which speaker. We're back in our `server.js` file of our Apollo server looking at our schema. Let's update it and add sessions below our `Speaker` types. Create a type, `Session`, give it some fields, create a type `SessionResults` just like our `SpeakerResults`. Add



a query that we'll make almost identical to our `speakersConcat` query, and let's call it `sessionsConcat`. While we're here, let's update our `Speaker` type to include returning sessions for a given speaker. Now let's move down to our resolvers and work on our `sessions` resolver. Because it's almost identical to our `speakersConcat` resolver, let's make a copy and include it here with some changes. Start out by changing the name to `sessionsConcat`. For our REST call, change `/speakers` to `/sessions`, and for `___` change last two eventYear. The next step is really not necessary for our pagination demo, but in the next module you'll learn about data loaders and their associated performance benefits, and for that we'll need to add a resolver to our `Speaker` type to return a list of sessions. All we have to do for that is add another type to our resolver. Currently, we have `Query` and `Mutation` types, now we'll have a `Speaker` type. At the field resolver for `sessions`, we'll only use the first parameter, `parent`, so declare that. Then, in the method, get the `speakerId` for the sessions we want to return. Then get from our REST server both the `Sessions` and the `SessionSpeakers` lists. Calculate a list of `sessionIds` based on our `speakerId` from the join list, then filter and sort our full sessions list by `eventYear`, and return the subset of sessions for the speaker. That's it. Let's run our Apollo GraphQL server now with `npm start`, and do a couple of queries. First, a list of sessions, passing an empty cursor and a limit of 3. That works. Next, let's add to a speaker query that we've run before the field sessions that is sessions for each speaker. Run that, and it works also, exactly as we'd hoped.

Generalizing Our Load-more Pagination the Apollo Way

If you carefully look at our Apollo type policy definition that gives us the load more style pagination that's here in our `apolloClient.js`, you might notice that there is nothing here specific to speakers. The only assumptions we make are that in your GraphQL query results you'll have a `datalist` property that contains your array and you'll have a `pageInfo` property that likely holds metadata about your results. This means we can easily extract this to its own file and reuse it for other datasets besides just speakers. Let's do that. Let's take a copy of everything assigned to the field `speakersConcat`, then create a new JavaScript file in our `graphql` folder called `pagination.js`, and in that export a generic function that we'll name `generalConcatPagination`. Let's have it return what we copied from our type def, and we're done. We can reuse this function now any place we want to implement load more pagination, assuming we follow the same coding style. Back to our Apollo Client. We just need to call this function, `generalConcatPagination`, and we're almost done. All that's left is to import it at the top. The Apollo team has created multiple pagination solutions in this style that are built into the Apollo Client library. Currently they have three of these written, including one that works with relay-style paging. You can read more about them here on the Apollo GraphQL website. Next up, we'll use our `generalConcatPagination` to do load more type scrolling with our new sessions query we added to our GraphQL server



in a previous clip.

Applying Our Generalized Load-more Paging to Our New Sessions Data

Now comes the really easy part. We've done all the hard work generalizing our load more style pagination, now all we need to do is implement our new sessions data type on our React client. Let's start in queries.js and add a new sessions query almost identical to our speakers query previously used. That is copy all of GET_SPEAKERS_CONCAT to a new query named GET_SESSIONS_CONCAT. Rename speakersConcat to sessionsConcat, change field names to title, eventYear, and cursor, and that's it. Let's create a new page to do our load more style paging for sessions. Make a copy of our pages/SpeakersCursor.js and call it SessionsCursor.js. And do the same for SpeakersLoadMore in our components folder. Make it SessionsLoadMore. In our new SessionsCursor.js, rename the Speaker mentions to Session, and then let's do a similar thing in our SessionsLoadMore.js file. First, change our query from GET_SPEAKERS_CONCAT to GET_SESSIONS_CONCAT. Rename LoadingMoreSpeakers to a more general LoadingMoreData. Change the view reference from data.speakersConcat to data.sessionsConcat. In the render, update id, first, and last to id, eventYear, and title. And that's almost it. The last thing is go to our apolloClient.js, create an almost identical query field to our speakersConcat for our sessions, just sessionsConcat: generalConcatPagination, and we're done. Open our running app, browse to the URL /SessionsCursor, and we can continually scroll through sessions. There's a lot, as we can see them by event year, so no need to go to the bottom of the list. We've reached the end of our Apollo Client pagination module. We've just scratched the surface of both the types and implementations that can be done for pagination. There's a standard out there called Relay, which a lot of companies use for pagination. Probably the most standard implementation of that is on GitHub. So, if you want to see it in action that's a good place to look. Just remember, there are lots of trade-offs and never a one-size-fits-all solution. Just keep in mind your goals and go from there

Using the DataLoader to Improve Server Response Times

Learn Why We Need a DataLoader

Hi. I'm Peter Kellner, and welcome back to my course covering Apollo performance management. GraphQL gives us enormous flexibility in how we can query our data. That flexibility often leads to very poor server



performance, leading to slow response times on clients, as well as high server costs for processing data. DataLoader goes directly after one of the biggest performance bottlenecks that typically occur inside a GraphQL server. That problem is typically referred to as the N+1 problem. The best way to explain this is through an example. In our GitHub repo that accompanies this course, the Apollo Server we have been developing through all the modules has the N+1 problem, and by the end of this module we will have solved it. If you want to follow along, download the source code from our GitHub repo at this URL and navigate to module 7 clip 1. There are two subfolders there, one is the React app named ApolloClientReactApp, and the other is our server just named ApolloServer. That's the one we will be working with first. To launch the server, bring up a terminal prompt in the ApolloServer folder, make sure you've run npm install, then enter the command npm start. That launches our REST server, which runs on port 5000, which we use as a source for our data and our Apollo GraphQL server running on port 4000. Browsing to our localhost port 4000, we have our GraphQL playground where we can send commands to our GraphQL server. To start, let's enter a query for speakers only. Execute it, and it comes up really fast. Taking a look inside our GraphQL server in server.js, we have our speakers query that returns to us SpeakerResults. Scrolling down to our speakers resolver, all we are doing is making one REST call for speakers and then returning the results. Let's prove that by bringing up our terminal window where we ran npm start alongside our playground. Executing the query again, we can see our REST GET /speakers call being echoed, just one. Now let's add sessions to our query and retrieve all the session titles for each speaker. Execute that. Wow, 20 REST calls. What happened? For each speaker, we are only showing 10 of them, the server made two REST calls. That's because in order to know the sessions for a given speaker we need to query our REST sessions and also another table, sessionSpeakers to get the join between our sessions and speakers tables since we can have multiple sessions per speaker. Let's go back to our VS Code source and look more closely at what's happening. Here's our speaker type. Notice that the sessions field is returning an array of type Session. Let's browse down to our speakers resolver, and specifically the sessions field resolver. All the other resolvers for sessions are generated automatically by the Apollo Server. Notice the first thing we do is get a reference to our parent.id field, that's our speakerId, then we make two REST calls, followed by a bunch of data manipulation, and finally return the sessions just assigned to this one speaker. This gets called for every speaker, which is why we had 20 REST calls. It's clear we don't want to call the sessions resolver for every speaker. And the obvious thing to do is to batch those calls and then make just one call that gets the sessions for all of our 10 speakers at once. This is exactly the case for the DataLoader. What the DataLoader does is instead of doing all this work in our sessions resolver it gives us a way to batch our sessions requests and then retrieve all the results at once. Each time a resolver is called a promise gets generated that will get fulfilled before our GraphQL server command is done completing execution. All the speaker requests are then batched



together, deduplicated, than in one pass, or two REST calls in our case, all these promises are completed and the data gets resolved. Once the request is done the results are lost. This is why DataLoader is referred to as a batcher, not a cacher. That's basically it. The rest is implementation details, and for the next few clips you'll learn those details. It feels a little tricky at first, but once you get the hang of it it's really easy and very worthwhile.

Add a Room Type and Make It a Field for Each Session Type

Retrieving multiple sessions for a given speaker using the data loader is a little bit tricky. Let's work our way up to that by first tackling a simple problem. That is, let's look at the problem which involves a one-to-one relationship. Each session is assigned a conference room. Since we don't have a room type in our schema, let's add it. Heading over to our schema, let's add it at the top. Give it three fields, ID, name, and room capacity. Then, scroll down to our session type and add it here also, just room of type Room. What we need to do now is add a room field resolver to our session type resolver. So far, all our other fields are automatically generated by the Apollo Server, so we need to add a new sessions resolver. Remember, this is a type resolver, so it's at the same level as the root-level resolver's query, mutation, and subscription. Add it, create an async room resolver. For now, we just need the first parameter, but for completeness, I'll add all four; parent, args, context, and info. First thing, let's create a convenience variable to hold the ID of our room field; that is, of the parent query, which in this case is sessions. That's `const roomId = parent.roomId`. Then let's call our REST server to get a list of all the rooms. Find the one room assigned to this session, and then return that. This is basically our complete room resolver for any given session. Back to our GraphQL Playground. Paste in a query for sessions at a field room, ask for name and capacity for each session, execute the query, and we got it. But looking over to the right, notice we called our REST service for rooms once for every session, and that's over 100 times. Coming up in the next clip, we'll solve this with the data loader.

Add a DataLoader for Retrieving a Room for Each Conference Session

Let's fix our retrieving of our rooms problem by using the Data Loader. Data Loader is an npm package, so first, install it. Npm install dataloader. Then in server.js, import it. Looking again at our Room resolver the third parameter is shared context. That's what's maintained by our Apollo server through the full GraphQL request. That's what Data Loader will use to solve our N+1, or over retrieving problem. Scroll down to where we instantiate and configure our Apollo



server. Let's add a new option, context, assign the context a function, and have that function return our named resolvers. In this case, let's name our Data Loader resolver `roomLoader`, new up the `DataLoader`, and give it an async function that takes in one parameter, a list of all `roomIds` we want to resolve. This is the part that's hard to understand the first time through, so let me stop here, `console.log` the `roomIds`, and just return. Let's go back to our room resolver, and then from the third parameter, context, destructure `roomLoader`. Let's replace all this code with one line, `return roomLoader.load`, and pass in `roomId`. What happens now on our behalf is that the `DataLoader` does not actually look up the room, like we did before, but instead, stores the `roomId` and returns to us a promise to fulfill that later. Think for a second about this. This is the full `DataLoader` pattern. We call our `roomLoader.load`, our `DataLoader` does not get the `roomRec`, but instead, stores the `roomId`, returns our promise instead, then when all the `roomIds` for the entire query has been requested, it asked for them all at once. That is the `DataLoader` pattern. Back to our playground. Paste in a query that gets us all our sessions, execute it, just one REST call to sessions and 106 sessions returned. Now, let's add our room with id, name, and capacity. Executing it again, it fails. No surprise. We didn't finish the code for our `roomLoader`. We just did a `console.log` of the `roomIds` passed in and returned. Looking at the console, here are all the `roomIds`, about 40 in all, covering the 106 sessions. That's because multiple sessions can happen in the same room and `DataLoader` deduplicated the list before returning to us. That's it, 1 batched call of 40 rooms. Let's finish up our `roomLoader` code now. Start out by getting all the rooms with our REST call. What we need to return from this method is very straightforward. We get passed in an array of `roomIds`. We need to return an array with the same number of room objects in the exact same order. Here is a debug view of what I mean. Notice, `roomId` is the array of integers that we got passed in and `x`, I just made a temporary variable to hold what we need to return, is a list in the exact same order, but instead of being an array of `roomIds`, it's an array of room Objects. Let's write the remainder of the code. Because we are going to want to look up each room by its `roomId`, let's start out by making a map of all the rooms. Then all we need to do to return what we want is map over our `roomIds` then return a lookup into our previously calculated map. Going back to our playground, re-executing our query, looking at our logs, only one call to sessions and one to rooms. Mission accomplished. Next up, we'll do the same thing, but instead of returning a single room for your speaker, we'll return a list of sessions. The strategy will be exactly the same, just a little more bookkeeping.

Add a DataLoader for Retrieving Sessions for Each Conference Speaker

We currently have this query which gives us a list of speakers along with all the sessions each speaker is given. This works, because in our schema we have defined sessions as an array of type `Session`. We currently have a resolver in our



Speaker type session that for each speaker makes a call to both sessions and sessionSpeakers. To solve this with DataLoader, it's a little trickier than what we needed to do our roomLoader, but not that much. Let's do it. Let's start out in our apolloServer config and add another loader. Let's call this one sessionsLoader, plural, since it brings back multiple sessions for Speaker. Same signature as our roomLoader, but the passed-in keys are speakerIds. We'll start out by getting both our REST data services, Sessions and a join table, SessionSpeakers. Now let's get a list of all the sessions that are associated with all the speakerIds passed in. We go after the join table and use the filter and include functions to do it. Finally, we do exactly what we did from our roomLoader. We map over our speakerIds, and after some computations, create a sessionsForSpeaker map that basically has a list of sessions for each speaker. Remember, a roomLoader just returned a one-room object. We want a list of speakers for each speakerId. The return is identical to roomLoader. We just map over our speakerIds again, then this time return the full object. All that's left to do is update our sessions resolver. This is identical to a room resolver. First, we just need to get rid of all the code except for our calculation of speakerId. We destructure from our context, sessionsLoader, then just return sessionsLoader.load(speakerId), and that's basically all we need for our sessions resolver. Now, back to the Playground, running our query again, just three REST calls total, not 21 like before. First our speakers call, then just one call for sessions and one call for sessionSpeakers. Mission accomplished. Next up, we'll include all this in our React UI.

Integrate Our Speakers Sessions and Rooms Our React UI

Let's take our accomplishments so far, that's adding both sessions and rooms to every speaker, and integrate those into our React UI. Let's also update our server to include a few more fields. Specifically to our server.js, let's add to our speaker type company, twitterHandle, and bio. Then back to our Apollo Client React app. Let's update our speakers query to include those three new fields, as well as a new sessions field that, in addition, includes the room name. Let's create a new landing page now that we can build our new example on. Rather than modify any of our existing landing pages in our pages folder, let's add a new one with the same format as the others and name it home.js. Have it render a new HomePage component. Let's go create that. Start with an empty component that just has our standard includes we've been using so far. At the top of our functional component, before the render, let's call useQuery to get all 10 speakers by default. Check our loading and error statuses, then some rendering code with Bootstrap CSS classes for styling. Notice we're calling a new component, SpeakerCard, which renders out each speaker. This component is new in just simple React code. We start out by sorting our passed in sessions by event year and limiting the display count to three per speaker. Then we render a nicely



formatted speaker card. Notice we are including a picture of each speaker, as our data is coming from a real conference I've been organizing for the past 15 years. We've got proper access to the images we can use. I could have downloaded them all locally for each clip, but that would have bloated our GitHub repo. Then, below each speaker profile detail we render our sessions, including the room for each session. Running our app now with `npm run dev`, everything works. Each speaker has multiple sessions and each session is in its own room. And meanwhile, both sessions and rooms are being loaded efficiently because our `DataLoader` code is in place. The takeaway from this module is that you view `DataLoader` as an essential tool to use to help make your GraphQL server process data more efficiently. Specifically, the problem it attempts to solve is the one where you repeatedly call items with their primary key when it would be more efficient to batch them up and then make one call to download them all. There's no magic with `DataLoader`, and it's even something you might have written yourself, but it's a nice pattern and is commonly used.

High Performance Caching Strategies with Apollo Server

To Cache or Not to Cache

Hi, I'm Peter Kellner, and welcome back to my course covering Apollo Performance Management. If you'd like to follow and/or code along with me, all the code for this course is in this GitHub repo. At the end of each clip, there's a folder representing the completed code. Just as side note, the `server.js` file, up until now, included all the code for our Apollo server. I have completely refactored it into about 10 separate files to make it more manageable. The functionality of the code is identical to what it was at the end of the previous module, just a lot easier to navigate. In this module, we will talk about strategies you can employ such that the data requested from your GraphQL server is available as fast as possible to the customer or user of that data. First, though, let me try to discourage you from caching any of your data when it's not needed. The reason, anytime you cache data, you create multiple versions of the truth. The question immediately becomes, is your cache a correct representation of your data? This leads me to claim, caching your data and what is the real truth of your data are enemies. Besides just getting to the truth, cache also slows you down when building your apps. When things don't work, you often have to swim through all your cache layers to figure out where the problem started. Adding new features means you have to take into consideration all of those layers and architect cache solutions around them, updating your apps becomes harder because there could be stale data out there, and you don't want your users to see errors, and that leads to having many more tests that are harder to write



because of all the added complexity. Bottom line, as Donald Knuth first said, "premature optimization is the root of all evil". Adding cache layers when you don't know that you need them might be considered premature optimization. With that disclaimer out of the way, let's talk about what caching we can do with our Apollo GraphQL server. Because ultimately, if your app is too slow, it will fail. Our assumption here is that you understand how well connected your data is to your GraphQL server itself. That is, say, for example, your server is reading directly from its own memory. That's a very different case than if your server were reading some ___ data from far, far away. To start, this is a scenario where we have no cache. That is, these user's React app sends a query to the GraphQL server, then the GraphQL server goes and gets the data from the original source, and finally, returns it as the response to the React app for processing in the browser. The thinking with caching data is that the closer to the user, the better. We'll build three scenarios, starting out with as close as possible to our user then move slowly away. To start, browser cache is the best. If the React app can request the data and get it served directly from the user's own browser cache without going over the internet, that is the ultimate win and fast. Next fastest is if we can get the data available in a Content Delivery Network, CDN, like Akamai, Amazon's CloudFront, or Azure. Those are geographically distributed, so hopefully, very fast across the internet. If you do have to go all the way to your GraphQL server for the data, then the cache plan there is to have it available from an internal cache to the GraphQL server. This can either be in the memory of the server, or more typically, to support web farm scenarios, some fast local database, like Redis or Memcached. To be clear, there are a lot more cache points available, but for the purpose of this course, these are the three that we will discuss and I'd estimate cover 95% of the scenarios you might run into.

Setting Cache Hints in Your GraphQL Server

Setting graph hints in your SQL schema are a way to tell it what it should cache, how long to cache it, and whether to allow intermediate caches such as proxy servers or CDNs to participate in that cache. Cache hints can be set on types also. They can be set in two different ways, either declaratively in your schema or imperatively in your resolvers. Let's start out by setting some cache hints declaratively around our Speaker type. When we assign the `cacheControl` directive to a type, that tells the GraphQL server to cache everything in that type. Passing in 3600 tells GraphQL to cache our speaker type for 3600 seconds, or 1 hour. By default, the scope of this is public, meaning all intermediate caches on the Internet can participate in this caching. That includes CDNs, proxy servers, browser cache, etc. Keep in mind, though, that unless you are using the Apollo GraphQL automatic persisted query feature with the option set to handle GET requests, all of your GraphQL traffic will be over POST. We'll dive deeper into that in a moment. Though the Internet protocol, RFC 2616 Section 9.5, says responses



to this message, POST that is, are not cacheable unless the response includes appropriate Cache-Control. However, many browsers and proxies will not cache POSTs, so it shouldn't be counted on. Later in this module, you'll learn how to change GraphQL queries to use the GET method, and that's cacheable everywhere, and the Internet is standardized for this. Continuing, if we don't want our Speaker favorite to be cached, say, for more than 5 seconds, and we don't ever want that cached publicly, that is, only let the user themselves store the data in their local browser, we set the scope to PRIVATE. Finally, maybe we want session types inside the Speaker's type to cache for only 10 minutes. To do that, we would set the maxAge to 600. That's a quick overview of setting your cache declaratively. There may be times when you want to set the cache headers imperatively, also referred to as programmatically, that is possibly if you were checking for authentication status. To do that, you use the same two parameters, maxAge and scope, and in your resolver use the fourth parameter, info, to set cacheControl. Lastly, and I would not recommend this, is if you want a default cache timeout for all of your data, you can set it when you initially configure your Apollo server by adding it as it passed in config property. By default, any data you cache will be stored and managed using the local in-memory cache provided by Apollo. Apollo has also provided multiple cache-backing data stores that you can use. They include both Redis and Memcached. It's best to use one of these if you are running in a web farm type scenario. As you'll see in a later module, if you are running serverless, meaning that every request launches the Apollo server, you'll have to use a non-in-memory based provider to make that work. Apollo lists on their site the currently supported configurations, along with links for how to set up those non-in-memory data providers.

The GET Method and Automatic Persisted Queries

Remembering that the internet is best at caching HTTP GET method requests, it makes sense that GraphQL should provide a way for us to take advantage of that, especially for a high volume site using GraphQL. There is currently nothing in a GraphQL spec for that, so Apollo has invented their own solution that involves their own protocol for making this happen. Lucky for us, if we use both an Apollo server and Apollo client, we have very little setup necessary to make this work. In the clip following this, I'll go through the details, as well as show you the network traffic associated with using this new Apollo feature. For now, let's talk about how this works from a high level, including the motivation for Apollo creating it. As you can imagine, as applications get more complex, the GraphQL query string that needs to be passed to the server can get quite large. These strings can be significantly longer than a GET request can handle, and since we know we want to be able to support GET requests, this is a major blocking point. Not only that, if we were to send these queries, it's a big waste of bandwidth since we are repetitively sending the same query. Apollo solved this



by inventing a new protocol that helped them to store queries from the client that could be referenced with the unique key without having to resend the full query multiple times. That is the first time the client sends a GraphQL query, the server remembers it in its cache and returns to the client the result of the query. What does this protocol actually look like? Let me explain by following a single query from a GraphQL client to the GraphQL server for the first time, then talk about subsequent requests. First, it's important to understand that the hash of a query is deterministic. That means we can take our query, run it through a hashing algorithm, and every time that query will generate the same hash value. We can't go the other way, however, that is, we can't take that hash value and turn it back into a query string, but we can know for sure whether the server has the query already in its cache. So here is the flow starting with the Apollo client. Just before the Apollo client would send a query, it calculates the hash for this query. It does what we call an optimistic request. This means it sends an HTTP GET call to the server hoping the server can process it. Assuming the server has never seen this request before, it returns to the client a rejection and does no processing. Then the client, after getting rejected, sends the full query back over a POST request to the server. The server executes the request, then responds with the full results. Meanwhile, though, the server calculated the hash of the request the same way the client does and stores the full query along with the hash for the next time. If all your requests are unique, which is highly unlikely, this is a bad plan because it took us two round trips to the server to get the response we wanted, however, likely you'll send the same request over many, many times so the next time you make the request, the path is just like this. Your client sends the request with just the hash, and since the server has the full request stored, it can execute it and return directly, just one request with a small payload over HTTP GET and the full result is returned. Again, like I said earlier, since this is over GET, it's easy to configure a CDN to cache the results for us, and then next time, our GraphQL server never even needs to see the traffic. Let's build this out in our next clip.

Implementing Automatic Persisted Queries for Best Caching

Because of the tight integration between Apollo Client and Server, the implementation details necessary to add Automatic Persisted Queries is quite simple. It turns out we do not need to make any changes to our Apollo Server code, just to our client. The Apollo Server has, by default, everything built into it we need. So, I've got our Apollo Client open here in VSCode. That's in the folder ApolloClientReact app. There are three node packages we need to install. To do that, enter the command `npm install, apollo-link-persisted-queries, apollo-link, and apollo-link-http`. Just a quick mention, though. At the moment, there's a dependency bug in the Apollo library that appears if you are running npm 7, so if you are either switched to yarn or install NPM version 6, and you'll be fine. Open



the file `apolloClient.js` and let's update it to include what's necessary to enable Automatic Persisted Queries. First, import our three new libraries at the top. Then, create a new link object by using the following `ApolloLink` import. Set `useGETForHashedQueries` to `true`, and that takes care of switching from `POST` to `GET` for us, and also, notice that if you are running on a `localhost` port other than `80`, you'll need to mention the fully qualified domain like I've done; otherwise, just `/graphql` is enough in the URI spec. Lastly, remove our URI from where we instantiate the `Apollo Client` and add the link object instead using `ES6` shorthand syntax. That's literally all we need to do. Let's launch our client app with `npm run dev`, then instead of `Chrome`, let's browse with `Firefox`, as its got a nicer network debugger interface. I've also brought our terminal window over for the server so you can see it at the same time as the `React` app running in the browser. Let's browse to the speaker's page we wrote previously. Notice we have an `HTTP GET` for our `GraphQL` command. Before that, it would have been a `POST`. Expand the header of the `GET` command. Notice the variables are passed in as before, but the query itself is just the `persistedQuery` hash, and it's not very long. Let's right mouse on the `GET` and resend it to our `Apollo GraphQL` server. Check out the response. It's got all our data because the server could look up the query based on the passed in hash. To prove this is working, let's stop and restart our `Apollo Server`. Just `CTRL+C`, then `npm start` in our terminal window. Now we send the query again, browse to the response, we get an error saying `PersistedQueryNotFound`. Exactly what we should have expected. The only way to get this query registered again is to refresh your browser, and our `Apollo Client` and `Server` will exchange messages behind the scenes for us and reregister the query. I hope you see like me that this new feature is a game changer for `GraphQL` when it comes to enterprise scale applications, because now, being able to leverage `HTTP GET` with `GraphQL`, along with persisted queries in our `Apollo Server`, we have now infinite scaleout available to us. The big takeaway for both this clip and the module is now with these advanced cache features, `GraphQL` has arrived as a significant player in the ecosystem of high performance websites. In the next module, we'll look at two other big potential game changers for `GraphQL`; server-side rendering your `React` app with `Apollo Client` and serverless deployments of your `Apollo GraphQL` server.

Improving Performance with Server-side Rendering and Serverless Computing

Serverless and Server-side Unrelated but Both Important

Hi, I'm Peter Kellner, and welcome back to my course covering `Apollo` performance management. I've put the categories server-side rendering and serverless computing together, because they are both performance-improving



technologies based on improving your customer experience. Serverless computing and how it relates to the Apollo GraphQL ecosystem is 100% about your server, and is completely invisible to the Apollo client. In reality, there is absolutely a server involved. It's just implemented with serverless in a different way, such that every request sent from your client is handled without any expectation that it will be processed by the same server. That, combined with the idea that while no requests are being sent to the server, there is no server process waiting or spun up idly, for those requests. Server-side rendering, on the other hand, is all about the Apollo GraphQL client and React working together, nothing to do with the GraphQL server. It involves having your full app running in your web server, running Node, and then that server completely figuring out what the rendered HTML should be, and send that down to your browser for immediate display. The big up side to server-side rendering is, if done right, the browser-based experience is awesome, because there are no delays in seeing your data or extra round trips between the browser and server before the data appears. It does require a lot of considerations, however, as now forever request the server has to potentially do a lot of work. In this module, we'll start out by talking in detail about serverless computing, and you'll learn clearly what the pluses and minuses are of using that solution, as there are quite a few vendor-specific details that need to be implemented to go serverless, we won't actually deploy a serverless solution to any particular vendor. We'll quickly move on to server-side rendering, and we'll completely build out that solution, including rendering our paginated speakers page. We'll also take a look at performance optimization that can be done with the Apollo GraphQL client such that the cache can be populated ahead of time so that browsing forward or back for however many pages you want to pre-load can be done with no server interaction. Let's get started first with serverless computing.

Serverless Computing and Apollo GraphQL

If you're not using serverless computing, likely you've got one or some fairly stable number of servers processing your requests for you. They likely all have multiple CPUs, a reasonable amount of memory, and for the most part are always available. Typically, each server is hosting hundreds, if not thousands of requests a second, and likely have some kind of common backbone or session key keeping them consistent so incoming users are not blocked to one server or another. In general, server farms try to maintain session affinity to keep users coming back to the same server for best performance. Serverless, on the other hand, is based on the premise that you are only going to visit a server for just one request, and then, effectively, that server is going to go away. If you have thousands of requests coming in a second, that means you may potentially have thousands of different servers processing your requests. The idea is that there is really no upper limit to your app's scale. It also means as a customer you only pay for what you need when you need it. The key to this working is that your app has to be



able to efficiently start up and shut down. For that to happen, all the servers have to be very closely peered, with an exceedingly fast database like Redis or Memcache, such that when your serverless instance starts, in just milliseconds, it could completely be ready to process a request. Ideal problems that target serverless are things that don't require any outside data to start. That is, if you were to build a server-based calculator, it could come up really fast, and then when the user is done with it, nothing to save. The Apollo GraphQL server is not so easy, but the good news is that the Apollo team built it with serverless in mind. There are built-in connectors that support easily both importing and exporting the complete state of the server. In short, here are the lifecycle steps of the server request. It starts with a GraphQL client sending a request to the server. The serverless host receives that request and allocates a compute container to process the request, then loads the GraphQL server. The server imports the current cache from the fast database. It then processes the request and returns the data as a response to the GraphQL client. The server container updates the fast database with any changes the request affected, and finally, the serverless container is deallocated, and we're done. The only thing left are the details to make all this happen. Looking at the Apollo docs here under Deployment, they outline how to deploy serverless to several top cloud providers, including Azure and AWS. Though the steps are quite different, they all basically do the same thing. Scrolling down to Deploying with Azure Functions, for example, near the bottom under Sample Code, notice they have us update what comes after we create an Apollo server. In our current app, after the Apollo server instance is created, we launch an HTTP listener and wait for requests. With a serverless installation, instead, a handler from the ApolloServer Azure functions npm package is used, and that effectively takes care of processing just a single request from an Azure function. I encourage you to go through the steps for one of these technologies and see for yourself if it's a good fit for your problem. I suspect we will all be doing a lot more serverless in the future.

Looking Ahead at Our Final Server-side Rendered App

Sometimes it's just easier to show you instead of telling you what we're going to build out. That will also help you understand our goal. To start, let's run the code in our GitHub repo shown here, that is, under the folder module 9 clip 1. First, you need to navigate to the ApolloServer folder, run npm install, then npm start. Next, navigate to the ApolloClientReactApp folder and run npm install and then npm run dev. We've now got our paginated speaker's interface running. Let's now browse to that running app. Notice we can go from page one to page two, and we see a loading message. Then to page three, same loading message, But going backwards, no loading message. That's because Apollo saved in its cache the previous page we visited. Let's do a view source. Copy that into VSCode and have a look at it formatted. Notice it's about 100 lines, and nowhere in this code is our actual speaker data. All that speaker data in HTML was created dynamically



by the downloaded JavaScript after this page's DOM was ready to run. To make this extra clear, I'll turn off JavaScript processing in our browser, refresh the page, not only no speaker data, no data at all. As expected. Let's time travel to the end of this module and run the code we will have completed. Reloading the browser, JavaScript is still off, yet the full page rendered. That's because all the data is in our browser. Let's turn JavaScript back on now. Continuing, do a view source, copy this into VSCode, and let's have a look at it. To start, it's over 1000 lines of code, a lot more than 100 from before. Scrolling down, here are the real speakers, Arun Gupta, Chris Richardson, Mark Abramson, Douglas Crockford. Scrolling down further, notice our NEXT_DATA section. What this is is a full dump of the Apollo cache after the page rendered. This page was completely rendered in our node server, so our server code created this big chunk of data that represents the full cache. What's particularly interesting is look at our ROOT_QUERY section. Notice that it's got cache for both our page one speakers, as well as our page two speakers. When we go back to our app, running from our server-side rendered data, pressing the page forward icon, no loading message. That's because our server code preloaded our current page and the next page in anticipation of a user going to the next page. Going forward again, we do get a loading message because we only preloaded one extra page of cache. Hopefully, now you're thinking, I want to build that. Let's do it.

Updating Our Apollo Client Custom Hook for Server-side Rendering

One thing that makes building server-side rendering in SPA apps hard is that they have to load all the data that the page normally would load asynchronously, like REST calls, inside Node then get all that data down to the client to do what's called rehydrating the DOM. Because of Apollo's design, exporting the Apollo cache inside Node is easy. Then reimporting it into the local running browser-based JavaScript app is also easy. That data is just embedded in the page source. One reason, besides it's really easy to use, I chose Next.js as our React framework is because it's built to handle server-side rendering well. Let's make the necessary changes now to our app. Start with our `apolloClient.js`. That's where we instantiate the Apollo Client. First, let's hoist our config object that we passed into our Apollo Client to the top of that file. Then add to our `useApollo` hook two parameters, `initialState` and `pageProps`. Next, rather than returning a newed up Apollo Client directly, let's instead return a new Apollo Client that we created by executing a new method, `initializeApollo`, that took in as parameters our initial cache state, as well as page props that may have also been passed in. The return is also memoized to avoid needless renders. And don't forget we need to import `useMemo` at the top. While we are here, let's add a new variable, `apolloClient`, that we'll be referencing later. Now, to code up our `initializeApollo` method. Declare it with two parameters, `initialState`, which we will default to null, and `pageProps` for passing through properties. Immediately create



a local reference to our Apollo Client, `_apolloClient` that is. If it's defined already, a previous call to this method, `initializeApollo`, may have set it and stored it to our instance of `apolloClient` we just declared. And if not, new it up like we had done previously to create it. Only difference now is we have a `ssrMode` we need to set to tell Next.js whether this code is running in Node or running in the local browser. Next, check if `initialState` was passed in and not undefined. Then extract what's currently in our `apolloClient` state, `cache` that is, and merge it with our `initialState`. Merge is a named export of the npm package, `deepmerge`, which we will need to install, `npm install deepmerge`, and import at the top. Then update `_apolloClient` with the new merged state. If we were running inside Node, return `_apolloClient` and we are done. If `apolloClient` doesn't exist then assign it to `_apolloClient` and then return that. I get this is a little confusing because the code is running in both the client and the server. It very much violates the single responsibility principle. However, it's not a lot of code, and I don't see a way around it. Now let's look at our `_app.js`. This is our higher order component that Next.js wraps all our components with. All that's needed here is to add parameters to our `useApollo` hook we just modified. Pass in our initial state, as well as `pageProps` that might be just passing through from our caller, such as query strings if they exist. We are now ready to update one of our landing pages. In the next clip we'll update our `pages/index.js`, which is where our paginated speakers page comes from.

Adding Server-side Rendering Functionality to Our Paged Speakers Page

Now that we've got all our basic utility code in place, let's update our app home page, `index.js`, to correctly work with server-side rendering. To do this on every page that needs external data, like speakers, we need to add and export a very special function that only runs in the Node server when the page loads. This function is `getServerSideProps`, and whatever it returns gets loaded as incoming properties to the main component in this file. The beauty of this approach is that whether this component is running on the server or it's running in the browser, the received props are going to be the same. In this function, let's create a new Apollo client by calling `initializeApollo` with no parameters. We have no previous cache we need to bring in here. This will end up assigning a new Apollo client to the variable `ApolloClient` and our `use Apollo` hook we created moments ago. Don't forget to import `initializeApollo` at the top. Then we set our current page to 1; remember, we started counting pages at 1 and not 0. From here, we query our GraphQL server with the identical GraphQL query that our speaker list item will call later when it's running in the browser. Notice we are calling `query` instead of `useQuery` like `SpeakerListItem` does. That's because we are running inside Node, and we want this call to query to run to completion and not to go through a loading state like it does in the browser. Let's fetch the next page of data also. So when the user browses to the next page, the cache is pre-



populated. If you wanted, this is where you could add logic to load multiple pages both behind and in front of our current page. And also important are `GET_SPEAKERS` from our queries folder. Finally, we return from here a props object that has our new, fully populated state that includes our two pages of speakers. That's it. We are done with all our code to make this page properly process our speakers page. Before running our app, let's take a look at the component `SpeakerListItems` that is the component we are calling, regardless of whether we are running in the Node server or in the browser. If you're still not sure why `useQuery` doesn't force another call to our GraphQL server, let's remember how `useQuery` works. Because our default network policy `useQuery` always looks in the Apollo client cache before going back to the GraphQL server for data. Because we carefully populated the cache inside our Node server with what is being loaded here, then downloaded that cache to our client before this page launched, the data is in cache. That's why there's no loading state and the `useQuery` never has to reach out to our GraphQL server. This pattern is really powerful and gives us a lot of flexibility in how we implement our server-side rendering apps. Now launching the app again, all the HTML got downloaded to the browser, and if you look carefully, we never saw a loading message when the page rendered. Paging forward to page 2, again, no loading message, because page 2 is already pre-loaded in cache. Page 3, yes, loading message, because we didn't load that into our cache. In the next clip, we'll add the case of passing a page number as a query parameter, and based on that, the Node server will load the correct page of speakers into cache.

Supporting Query Parameters Like Page Number Passed to Our Server-side Rendered Pages

It's important to not only server-side render unique landing pages, but to also render them based on parameters passed in on the URL. Let's time travel and jump to the completed code at the end of this clip, and then you'll understand exactly what I mean, as well as our goal. Let's run our app and type into our browser `localhost3000/?page=2`. If we turned off our JavaScript we'd get exactly the same thing, our page rendered with the appropriate data from page 2. Now, if we page forward, notice the URL changes to `page=3`. What's important to take note of is Page 3 loaded 100% from cache because we always preload the next page into Apollo cache. Press the first button to load Page 1. That wasn't in cache, so the app did need to fetch the data from the server, but still not a full page load. Just the new data retrieved and the URL changed to `page=one`, forward again and again, both times no loading message. Now, if we reload our browser by putting our cursor in the URL field and press Enter, sure enough, we get a server-side render of Page 3. Moving to our source code now, let's start on our landing page JavaScript. That's `pages/index.js`. Looking specifically at our code running in the Node server, that's our function, `getServerSideProps`, we can destructure from the context, which is passed in as a receiving prop, the property `query`. The



parameter page is just a property of query. Let's grab our query parameter page and send it to the constant current page. Current page will help us in the graphic URL queries below to preload the correct page data, but not help us to let the client app know what page we're on. If you remember initializeApollo takes a second parameter page props, which I said can be used to pass properties through. This is exactly what I was talking about. Let's move our initializeApollo call to after we calculate our current page, and then pass as the second parameter a props object, which is basically just an object with one property, current page. Last thing, since current page really is now a state of the app, let's add that to our props that we return at the end. So let's talk about the next problem. Let's say we pass in localhost:3000/?page=2. When the page renders everything on that page needs to know what the current page is. That is, for example, when the code in the node server process renders the paging toolbar, that toolbar shows the current page number, which in this case should be 2. Looking at the pagination toolbar component, notice our current pages retrieved from our pagination data, which we get from our useReactiveVar based on paginationDataVar. Remember that this is the mechanism Apollo client uses to manage local state. We somehow need to get our passed in query parameter page into our paginationDataVar. Turns out we have the perfect place to do this. Going back to our initialize Apollo function in our Apollo client.js, the second parameter passed in is our page props. We just set that from our index.js to include current page. So all we need to do is right here, update our paginationDataVar like we always do. Call it passing in our original data spread, then replace current page with pageProps.currentPage - 1. No need to import paginationDataVar because this is where we created it initially. Now when our code runs on the node server, the page will be set correctly in state and in the client just the same. Let's try it. Browse to localhost:3000/?page=2. Sure enough, our Page 2 of data, including the right page number showing in our pagination toolbar. Are we done? Maybe not. What happens when we press the page forward icon? The data changed, but unfortunately the URL did not. This is a completely different problem, but still easy to fix. It's 100% in the client code. We just need to change the URL to reflect the correct page without actually calling the server. Going to our paging offset limit toolbar component. We need to use the next router, import that at the top and then create a variable router from useRouter. In each on-click event, we need to call router.push to set the URL to the correct page. Scrolling down to our next page button, just add router.push with appropriate parameters instead of to currentPage + 1. Rather than doing this on all four buttons, let's replace this push and also the actual updating of the paging state with the pagination dataVar, that is, with a simple function call and name it processPage Change. That just becomes this now in our render section, then update all the on-click events with a call to process page change passing in the appropriate page. That should be it. Browsing to our app again, pressing Forward. Sure enough, the URL updates as we'd expect.



Module and Course Takeaways

Both serverless computing and server-side rendering of apps written using the Apollo ecosystem can bring great benefits to both your customers running browser apps, as well as the infrastructure folks who are responsible to get data delivered through all the various internet gateways and connections. For the most part, serverless computing is a DevOps thing as there's not much there to do besides follow the set of procedures provided by cloud vendors. Serverless is gaining in popularity very quickly, so if you're not using it I strongly suggest you at least test your Apollo Server apps with it. Server-side rendering also has great benefits. Browser users will get significantly faster response times as now as soon as the first GET request is processed for a URL immediately on response the browser user sees a result. No waiting for JavaScript libraries to download, call web services to get data, and then finally display the results. Also, because all the HTML is able to be crawled by search engines, your site SEO is significantly improved. Regarding the full course, I hope you learned a lot about Apollo and how both the Apollo Server and Client bring benefits that will directly affect both your programming productivity, as well as the performance of your entire app flow from server to client. I'd love to get feedback on this course. Feel free to post here in the discussion forum at Pluralsight. Or if you have issues with any demos you can post those on the GitHub repo. I also welcome pull requests. Stay safe.